RAKTSETU

Expiry-Aware Blood Redistribution Network

Final Project Report · Object-Oriented Programming in Java

Submitted by

Pritika Khandelwal - R037

MBA Tech (Artificial Intelligence)

Mukesh Patel School of Technology Management and Engineering (MPSTME)

October 2026

Live application: https://raktsetu-ubvx.onrender.com

GitHub: https://github.com/pritikakhandelwal/raktsetu

Contents

Contents 2
Abstract 4
1. Introduction 5
1.1 Problem 5
1.2 Gap 5
1.3 Objectives 5
2. Requirements 6
2.1 Users and roles 6
2.2 Functional requirements 6
2.3 Non-functional requirements 7
3. System design 8
3.1 How a change is saved 8
3.2 Database 8
3.3 Security 8
3.4 Deployment 8
3.5 National facility directory 8
4. Object-oriented design 9
4.1 The four pillars 9
4.2 Interfaces 9
4.3 Design patterns 10
4.4 Exceptions 10
4.5 Other Java features 10
4.6 Lambda expressions and functional interfaces 11
5. UML design documents 13
6. Implementation 30
6.1 Allocating a request (`RequestService`) 30
6.2 Expiry risk (`ExpiryRiskAnalyzer`) 30
6.3 Transfer planning (`AbstractRedistributionStrategy`) 30
6.4 Tamper-evident audit log (`AuditHashing`) 30
6.5 Urgent alerts (transactional outbox) 30
6.6 Concurrency (`NetworkLock`) 31
7. Testing and quality 32

How quality is enforced 32
8. Performance 34
8.1 Engine (JMH benchmarks) 34
8.2 Server (HTTP load test) 34
9. Research results 36
9.1 Set-up 36
9.2 Redistribution 36
9.3 Forecasting 36
9.4 Supporting research 37
10. Application snapshots 38
11. Program output 44
12. User guide 48
12.1 Build and run 48
12.2 First use 48
12.3 Optional settings 48
13. Limitations, future work and conclusion 49
13.1 Limitations 49
13.2 Future work 49
13.3 Conclusion 49
References 50
Appendix 52
Appendix A: Project structure 52
Appendix B: Console application menu 52
Appendix C: Test results by module 53
Appendix D: Database migrations 53
Appendix E: Technologies used 53
Appendix F: Source code and links 54

Abstract

RaktSetu finds blood units that are about to expire unused and moves them, with staff approval, to the centres and hospitals that can use them. Blood centres in India discard large volumes of expired components, and platelets last only 5 days. Existing systems such as e-Raktkosh show where stock is, but not which units will be wasted or where they should go.

The project is built in Java 21 as seven modules (a domain model, a decision engine, file persistence, a digital-twin simulator, a 16-option console application, a Spring Boot web server and a benchmark suite) plus a React front end. It supports three roles (network administrator, blood-centre staff, hospital staff), records every stock movement on a SHA-256 hash-chained audit log, and sends urgent SMS or WhatsApp alerts.

On a simulated Pune district network, expiry-first redistribution cut expired units by about 60% and halved unmet demand over 120 days. Profiling-led tuning made the engine 5 to 66 times faster at 1,000 blood centres, and the server handles 4.4 times more requests than before tuning, with no errors at 200 simultaneous users. The project is tested by 240 automated tests (234 in Java, 6 in the web front end), with at least 80% line coverage enforced on every module. The web application is deployed and publicly reachable at https://raktsetu-ubvx.onrender.com, with a searchable directory of about 52,000 Indian hospitals and blood centres.

1. Introduction

Blood is wasted mostly because it sits in the wrong place: one centre holds units it cannot use before they expire while another, a few hours away, runs short.

1.1 Problem

- Each component has a short, fixed shelf life: platelets 5 days, whole blood 35, red cells 42, plasma a year.
- Supply follows donation camps and routine dispatch, not where demand is. Rural centres often receive more than their hospitals use.
- Stock reports from centres are often days out of date, so a planner cannot trust what it sees.
- Blood collection in India rose from 126.95 lakh units in 2023 to 157.14 lakh in 2025, while discards rose from 8 lakh to 17.43 lakh; expiry was the most common reason, at about 46% of discards [8].

1.2 Gap

e-Raktkosh, India's national blood bank system, shows how much stock each blood bank holds. It does not predict which units will expire unused, and it does not recommend where they should go.

1.3 Objectives

1. Model blood components, facilities, stock and compatibility with object-oriented design.
2. Identify units at risk of expiring unused from stock levels and recent usage.
3. Allocate emergency and routine requests from the nearest compatible stock, soonest-expiring first.
4. Plan transfers of at-risk units for staff approval, comparing more than one strategy.
5. Keep a tamper-evident record of every movement.
6. Deliver it as a working multi-user web application, and measure its effect in a digital-twin simulation.

2. Requirements

Three kinds of user share one network, and each sees only what their job needs.

2.1 Users and roles

Role	Works at	Can do
Network administrator	The whole network	Set up the network; add facilities and people; plan, approve and reject transfers; see every request; read the audit log and alert status
Blood-centre staff	One hub or storage centre	Log collections (with the bag's donation number); report daily stock; dispatch requests; receive transfers; discard damaged units
Hospital staff	One hospital	Raise emergency or routine requests; cancel before dispatch; confirm receipt; check network availability and compatibility

2.2 Functional requirements

1. **Stock:** record units by component, blood group and collection date; work out expiry from shelf life; remove expired units daily as wastage.
2. **Compatibility:** apply red-cell and plasma compatibility rules (plasma reverses them); issue O- to non-O- patients only as a last resort.
3. **Allocation:** serve a request from centres within 90 minutes, nearest first for emergencies and exact group first for routine requests, soonest-expiring units first; all or nothing.
4. **Expiry risk:** flag units unlikely to be used before they expire, from stock and forecast demand.
5. **Transfer planning:** propose transfers of at-risk units to centres expected to use them (Nearest-First and Expiry-First strategies); staff approve each one.
6. **Data freshness:** treat a centre whose stock report is over 3 days old as unverified: search it last and send it no transfers.
7. **Audit:** record every collection, issue, transfer and discard on a hash-chained log that detects edits.
8. **Accounts:** first-run setup of an administrator; one-time passwords; forced password change; account lockout after repeated failures.
9. **Notifications and alerts:** in-app notifications for each workflow step; SMS or WhatsApp alerts for urgent ones.

10. **Sign-up with approval:** blood-centre and hospital staff can request an account; the network administrator approves or rejects each request and links the person to their facility.
11. **Facility directory:** staff pick their hospital or blood centre from a national directory, or enter it themselves if it is not listed.

2.3 Non-functional requirements

Quality	Requirement	How it is met
Correctness	Business rules verified automatically	240 tests; 80% line-coverage gate per module
Security	Session sign-in, CSRF protection, role checks, lockout	Spring Security; 429 after 5 failed sign-ins in 15 minutes
Integrity	Tampering with history is detectable	SHA-256 hash chain; health check reports DOWN on tampering
Performance	Interactive at state scale	Emergency allocation 0.13 ms at 1,000 centres
Usability	One command to run; no demo data	Single JAR with the web front end; first-run setup page
Portability	Runs on Windows, macOS, Linux	Java 21+; embedded H2 database or PostgreSQL
Availability	Reachable from any browser, including phones	Deployed on Render with a Neon PostgreSQL database; responsive layout with a phone menu

3. System design

RaktSetu is a modular monolith: seven Maven modules and a React front end share one domain model and one decision engine, so the console app, the web app and the simulator all make the same decisions.

The module dependencies are drawn in the package diagram in Chapter 5 (Figure 5.2).

The domain model has no dependencies; the engine depends only on it; everything else sits on top.

3.1 How a change is saved

The server keeps the network in memory for speed and treats the database as the source of truth. Every change runs through one method: the engine applies it, the audit events it produced are hash-chained and written, and the database tables are updated, all in one transaction. If saving fails, the in-memory network is reloaded from the database, so the two never drift apart.

3.2 Database

Flyway migrations V1 to V12 create and evolve the tables: facilities, routes, units, usage, wastage, the audit log, requests and their units, transfers, notifications, users and the alert outbox (V1 to V5), privacy controls (V6), account requests (V7) and the national facility directory (V8 to V12). The embedded H2 database needs no installation; PostgreSQL is used in production and in CI.

3.3 Security

- Session sign-in with BCrypt-hashed passwords; no default or demo accounts.
- CSRF protection on every change (token in the `XSRF-TOKEN` cookie, echoed in a header).
- Role checks on every endpoint; centre and hospital staff act only for their own facility.
- One-time passwords for new and reset accounts, with a forced change at first sign-in.
- Lockout after 5 failed sign-ins per account or 20 per address in 15 minutes.
- Phone numbers are masked in logs and screens; provider keys come only from environment variables.

3.4 Deployment

The web application runs as a Docker container on Render (Singapore region) and stores its data in a Neon PostgreSQL database. Every change merged into the main branch on GitHub is built, tested and deployed automatically, so the live site always matches the code. The free plan puts the server to sleep when idle; the first visit after a quiet period takes about a minute to wake it.

3.5 National facility directory

When staff request an account, they choose their workplace from a directory of about 52,000 facilities: 2,408 blood centres and 6,504 government hospitals from the National Health Portal (Government of India open data, through Esri's Living Atlas), and about 43,000 hospitals, including private ones, from OpenStreetMap (ODbL licence). If a facility is missing, the person can enter it themselves and the administrator adds it.

4. Object-oriented design

Every OOP concept in the course appears in working code, and each was chosen for a reason in the blood-bank domain, not as an exercise.

4.1 The four pillars

Concept	Where in the code	Why it fits
Abstraction	`BloodComponent` and `Facility` are abstract classes	Code works with "a component" or "a facility" without knowing which kind
Encapsulation	`Inventory` keeps each component and group in a private, always-sorted `TreeSet`; `SlidingWindowAttemptPolicy` hides its failure queues	Stock can only change through methods that keep it in expiry order
Inheritance	`WholeBlood`, `RedCellConcentrate`, `Platelets`, `FreshFrozenPlasma` extend `BloodComponent`; `HubBloodCentre`, `StorageCentre`, `Hospital` extend `Facility`; all business exceptions extend `RaktSetuException`	Shared state (ID, group, dates) is written once
Polymorphism	`getShelfLifeDays()`, `getStorageCondition()`, `canStore()`, `canCollect()`	Expiry is computed the same way for every component: 5, 35, 42 or 365 days come from the subclass

4.2 Interfaces

Interface	Implementations	Purpose
`AuditTrail`	`AuditLogger` (hash-chained file), `InMemoryAuditTrail`, `CapturingAuditTrail` (database), `NoOpAuditTrail` (benchmarks)	The engine records events without knowing where they are stored
`Repository<T extends Identifiable>`	`CsvRepository<T>`	One generic class stores any identifiable entity
`RedistributionStrategy`	`NearestFirstStrategy`, `ExpiryFirstStrategy`	The administrator picks a planning policy at run time
`DemandForecaster`	Moving average, exponential smoothing, Croston/SBA, day-of-week, seasonal smoothing	The risk check and planner ask "a forecaster" for demand
`AlertChannel`	`SmsChannel`, `WhatsAppChannel`, `LogChannel`	Adding an SMS provider is one new class
`LoginAttemptPolicy`	`SlidingWindowAttemptPolicy`	The lockout rule can be replaced

		without touching sign-in
`Identifiable`, `ExpiryTracked`, `Transfusable`, `Exportable`, `DistanceProvider`, `StockDataSource`	Domain classes	Small single-purpose contracts (interface segregation)

4.3 Design patterns

- **Strategy:** redistribution strategies, demand forecasters, alert channels, login-attempt policies.
- **Template Method:** `AbstractRedistributionStrategy.plan()` is `final` and fixes the algorithm; subclasses supply only `order()` and `choose()`. `HttpAlertChannel.send()` shares timeouts and error handling while SMS and WhatsApp build their own requests.
- **Factory:** each `DemandForecaster` creates `Forecast` objects (the records `Flat` and `Weekly`); `ComponentType.newUnit()` and `FacilityType.newFacility()` create the right subclass.
- **Repository:** `Repository<T>` for CSV files and the repositories in the server's `db` package.
- **Transactional outbox:** an alert is saved in the same database transaction as its cause and sent later, with retries.
- **Dependency injection:** collaborators arrive through constructors, so tests pass a fixed `Clock` and fake providers.

4.4 Exceptions

A checked hierarchy carries business rules: `RaktSetuException` with subclasses `InsufficientStockException`, `IncompatibleBloodException`, `UnitExpiredException`, `FacilityNotFoundException` and `InvalidDataException`. The web layer maps each to a clear JSON error and HTTP status. Two exceptions carry state that drives behaviour: `TooManyAttemptsException.retryAfter` becomes the `Retry-After` header, and `AlertDeliveryException.retryable` decides whether an alert is retried.

4.5 Other Java features

- Generics with bounds (`Repository<T extends Identifiable>`), records for immutable values, enums with behaviour (`BloodGroup`, `ComponentType`).
- Collections chosen for their cost: `EnumMap`, `TreeSet` (stock in expiry order), `ArrayDeque` (sliding window), `ConcurrentHashMap` (thread-safe lockout).
- Concurrency: a readers-writer lock around the shared network, `volatile` fields, scheduled jobs, and virtual threads in the load test.
- Lambdas and functional interfaces: the expiry order is a lambda comparator (`BloodComponent.FEFO = (a, b) -> ...`); `NetworkLock.Action<T>` is a `@FunctionalInterface`, so every web request passes its work to the lock as a lambda (`lock.writeSafely(() -> { ... })`); the strategies sort with method references such as `Candidate::unit`.
- Streams, file I/O for CSV and the audit file, and SHA-256 hashing.

4.6 Lambda expressions and functional interfaces

A lambda expression is a short, unnamed function that can be passed around like a value, for example `(a, b) -> a.compareTo(b)`. Java allows a lambda wherever a functional interface is expected: an interface with exactly one abstract method, such as `Comparator<T>`, `Runnable` or one of the project's own. RaktSetu uses about 280 lambdas, 100 method references (`Class::method`) and 92 stream pipelines in its main code.

Where	Lambda or method reference	What it does
`BloodComponent.FEFO`	`(a, b) -> { ... }`	Comparator that orders units first-expiry-first-out; the inventory's `TreeSet` uses it, so stock is always sorted soonest-expiring first
`NetworkLock.Action<T>`	`() -> { ... }`	Project's own `@FunctionalInterface`; every web request hands its work to the read or write lock as a lambda
`ExpiryFirstStrategy.order()`	`Candidate::unit`, `c -> c.source().getId()`	Sorts at-risk units by expiry, then by centre, with a chained comparator
`ExpiryFirstStrategy.choose()`	`Destination::spareDemand`, `(Destination d) -> d.route().travelMinutes()`	Picks the destination with the most spare demand, then the shortest trip
`NetworkService.summary()`	`r -> r.atRisk().size()`	Stream that adds up the units at risk for the overview page
Benchmarks and load test	`f -> !f.canStore()`, `k -> new ArrayList<>()`	Filters facilities and builds maps on demand with `computeIfAbsent`

1. A comparator written as a lambda (`BloodComponent.java`). The whole expiry rule of the inventory is one lambda: compare expiry dates, and break ties by unit ID so the order never changes between runs.

```
public static final Comparator<BloodComponent> FEFO = (a, b) -> {
    int byExpiry = a.getExpiryDate().compareTo(b.getExpiryDate());
    return byExpiry != 0 ? byExpiry : a.unitId.compareTo(b.unitId);
};
```

2. A functional interface of our own, called with lambdas (`NetworkLock.java` and `NetworkService.java`). `Action<T>` has one abstract method, so any block of code can be passed to `read` or `write`. The lock is taken, the lambda runs, and `finally` always releases the lock. Taking and releasing the lock is written once; each caller supplies only its own work.

```
@FunctionalInterface
public interface Action<T> {
    T run() throws RaktSetuException;
}
```

```
public <T> T read(Action<T> action) throws RaktSetuException {
    lock.readLock().lock();
    try {
        return action.run();          // runs the lambda passed in
    } finally {
        lock.readLock().unlock();
    }
}

// a caller: the overview page reads the network under the lock
return lock.readSafely(() -> {
    int atRisk = engine.risks(network, today).stream()
            .mapToInt(r -> r.atRisk().size())
            .sum();
    ...
});
```

3. Method references and chained comparators (`ExpiryFirstStrategy.java`). `Candidate::unit` is a method reference, a shorter form of the lambda `c -> c.unit()`. Comparators are combined with `thenComparing`, so a multi-level sort reads as one sentence.

```
sorted.sort(Comparator.comparing(Candidate::unit, BloodComponent.FEFO)
        .thenComparing(c -> c.source().getId()));

return options.stream()
        .max(Comparator.comparingInt(Destination::spareDemand)
            .thenComparing(Comparator.comparingInt(
                (Destination d) -> d.route().travelMinutes())
                .reversed()))
        .orElseThrow();
```

Lambdas keep this code short and close to the rule it expresses ("soonest expiry first", "most spare demand, then shortest trip"), and they let one method, such as the lock, work with any piece of code passed to it.

5. UML design documents

Eight PlantUML diagrams describe the system (drawn with PlantUML, a text-based UML tool); they live in `docs/uml/`, are reviewed like code, and are updated in the same change as the code they describe.

Diagram	File	What it shows
Use cases	`use-case.puml`	The three actors and what each can do, plus the simulator
Packages	`packages.puml`	Module dependencies: domain at the bottom, server and console on top
Domain classes	`domain-classes.puml`	Components, facilities, inventory, requests, transfers, interfaces and exceptions
Engine classes	`engine-classes.puml`	Allocation, expiry risk, freshness, the strategies (Template Method), transfers
Emergency request	`sequence-emergency-request.puml`	Sequence of an emergency request from the hospital to issued units
Security and monitoring	`server-security-monitoring.puml`	Sign-in lockout (Strategy), health check, metrics, the readers-writer lock, incremental audit check
Urgent alerts	`alerts.puml`	Alert channels (Strategy and Template Method), the transactional outbox and the dispatcher
Demand forecasting	`forecasting.puml`	The forecasters, the `Forecast` objects they create, and where the engine uses them

The following pages show the diagrams one per page, each filling the page. The larger class diagrams are split into focused views (for example, the domain model into components, enums, facilities, transfers and exceptions) so that the text stays large. Wide diagrams are turned sideways on the page; turn the page to read them.

Figure 5.1: Use cases: the three actors and what each can do, plus the simulator

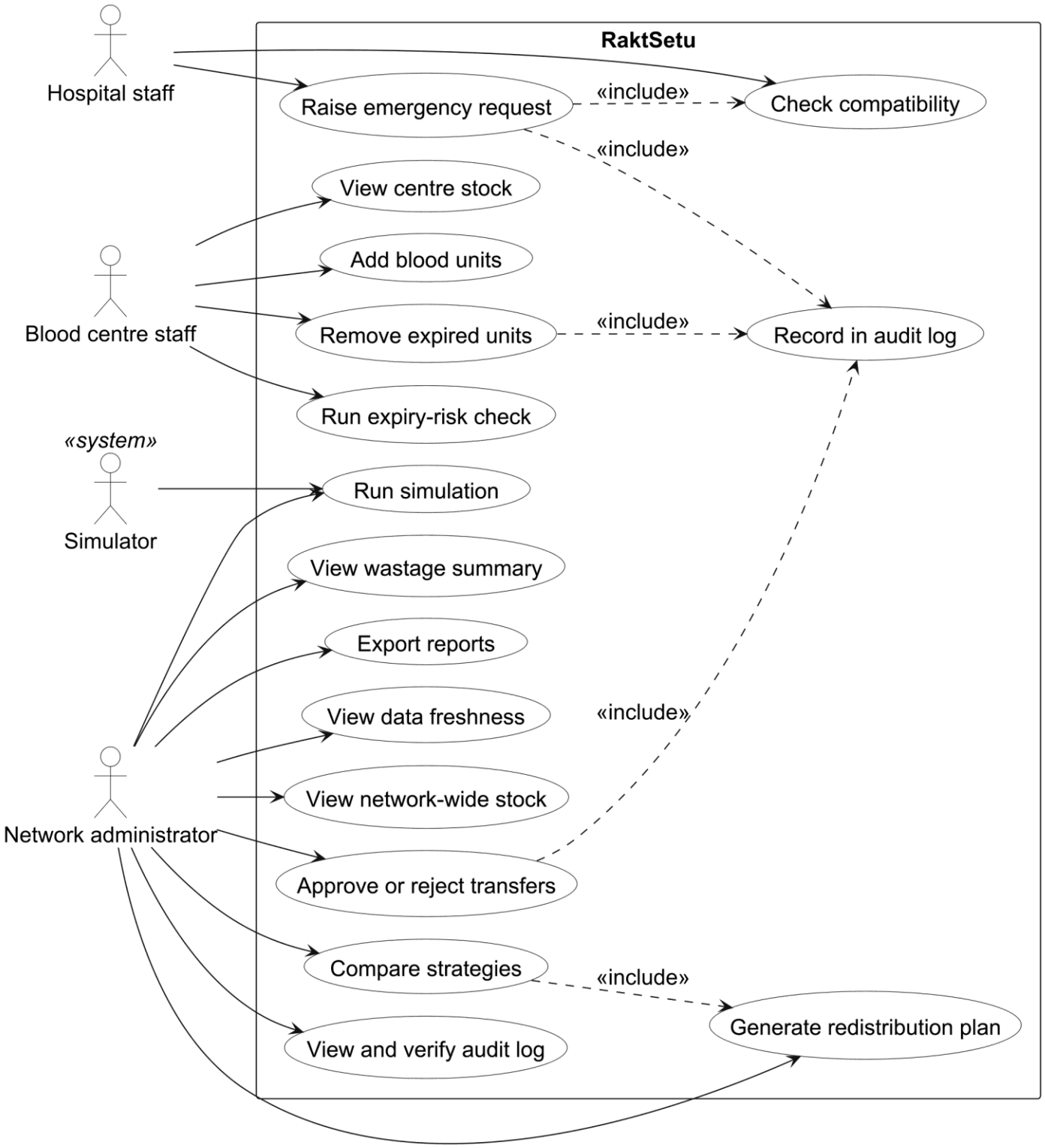


Figure 5.2: Packages: module dependencies, with the domain model at the bottom

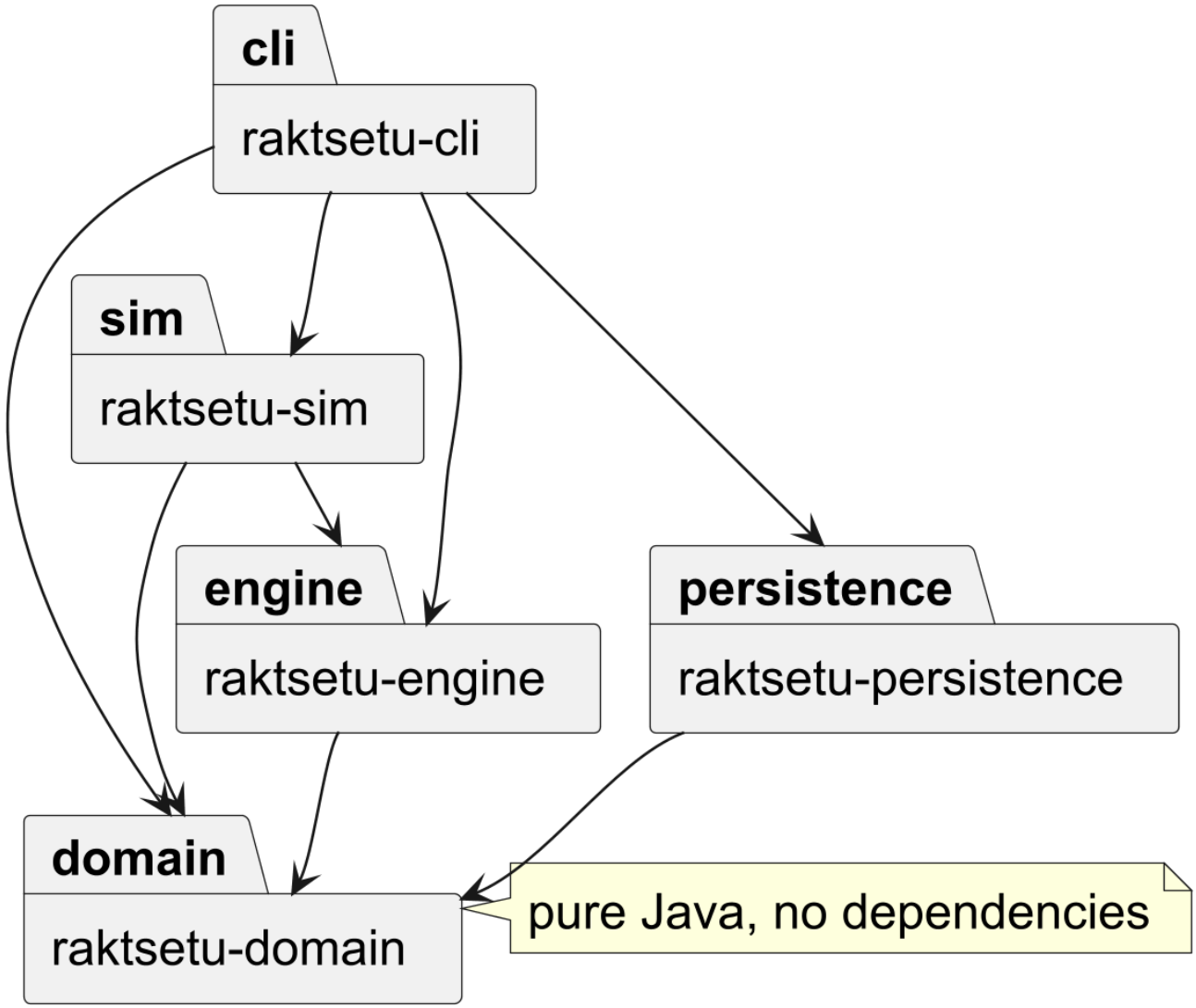


Figure 5.3: Domain classes: the abstract BloodComponent, its interfaces and its four subclasses (turn the page)

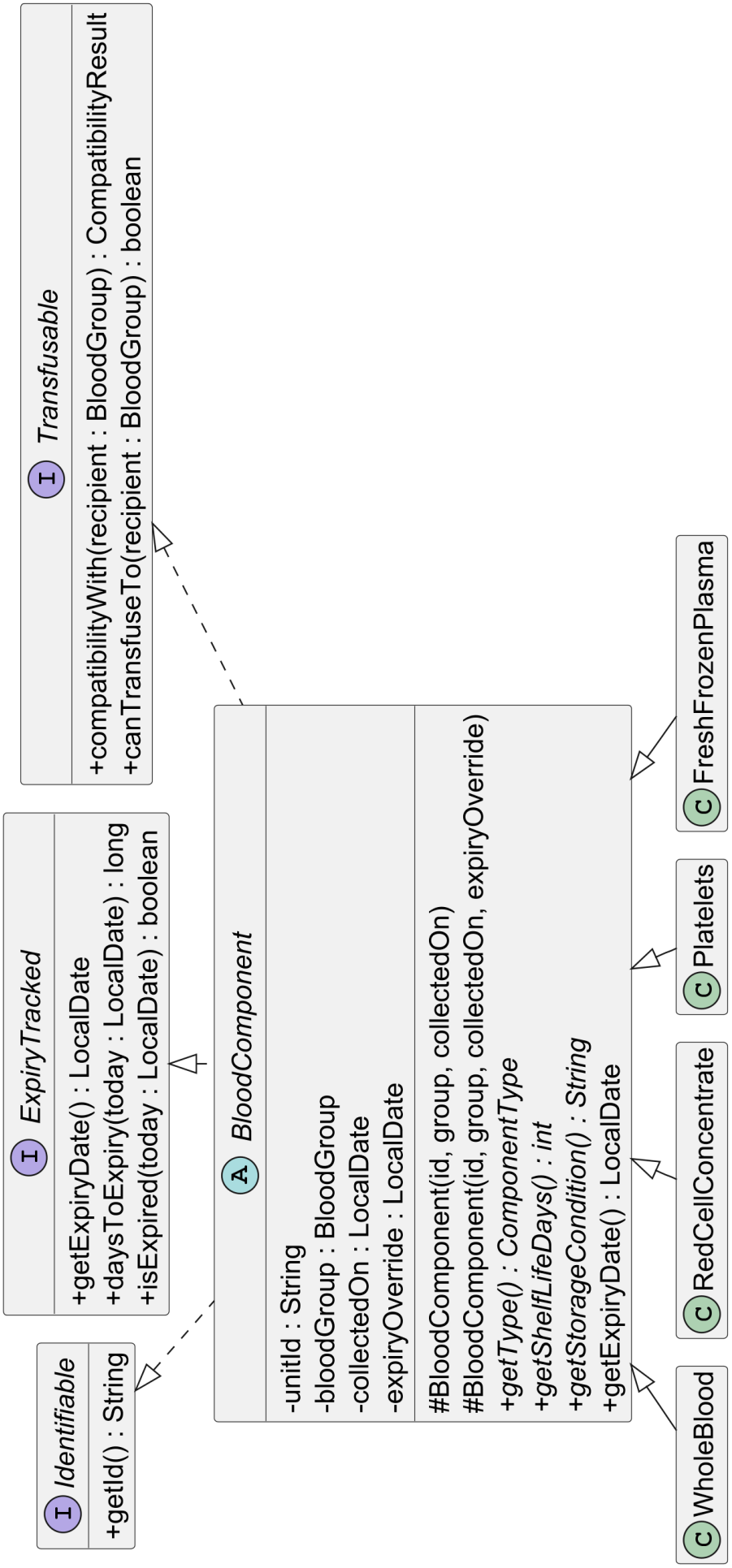


Figure 5.4: Domain classes: blood groups and component types (enums with behaviour, factory method)

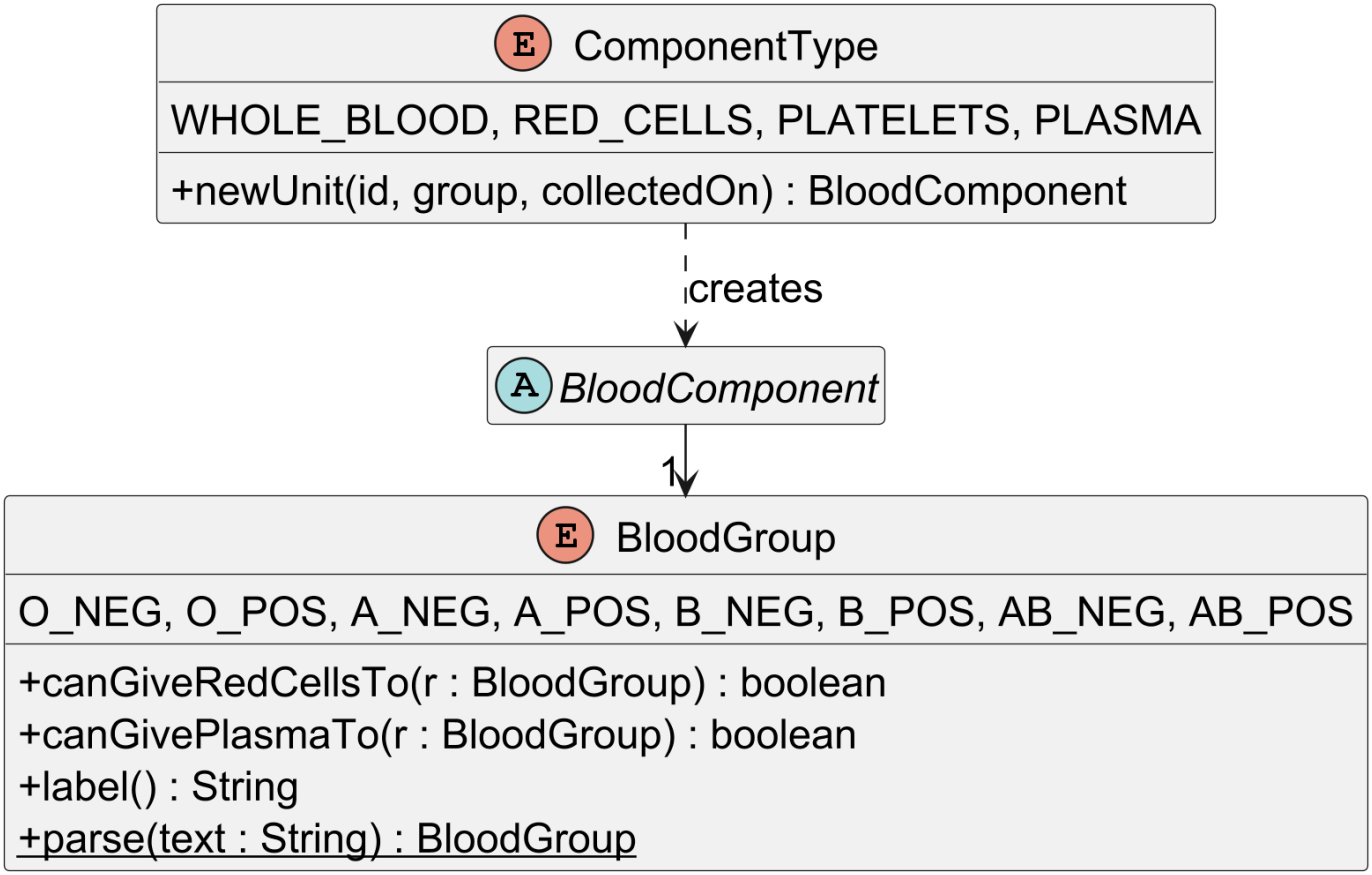


Figure 5.5: Domain classes: facilities and their inventory (composition) (turn the page)

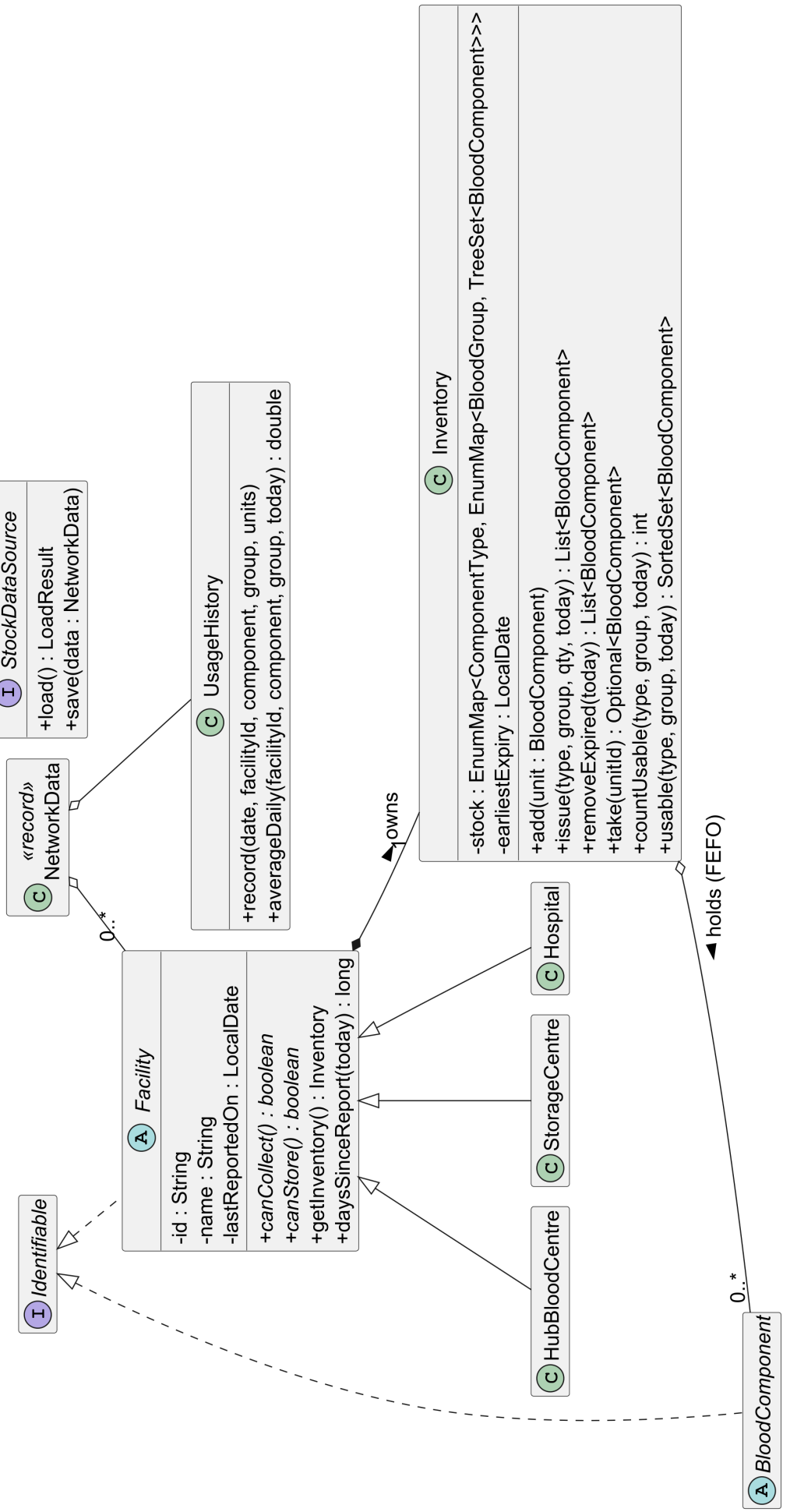


Figure 5.6: Domain classes: requests, transfers, routes and records (turn the page)

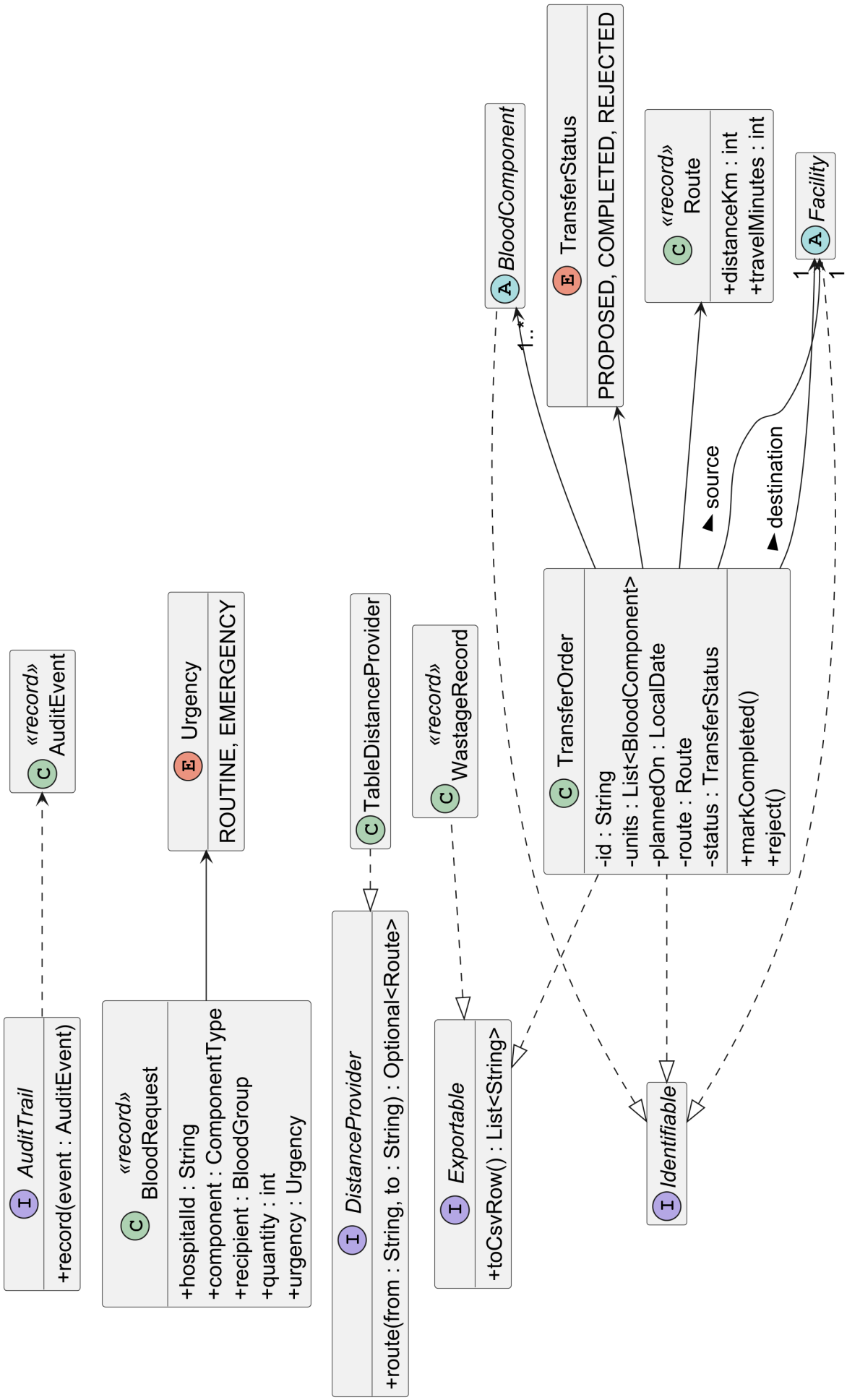


Figure 5.7: Domain classes: compatibility results and the checked exception hierarchy (turn the page)

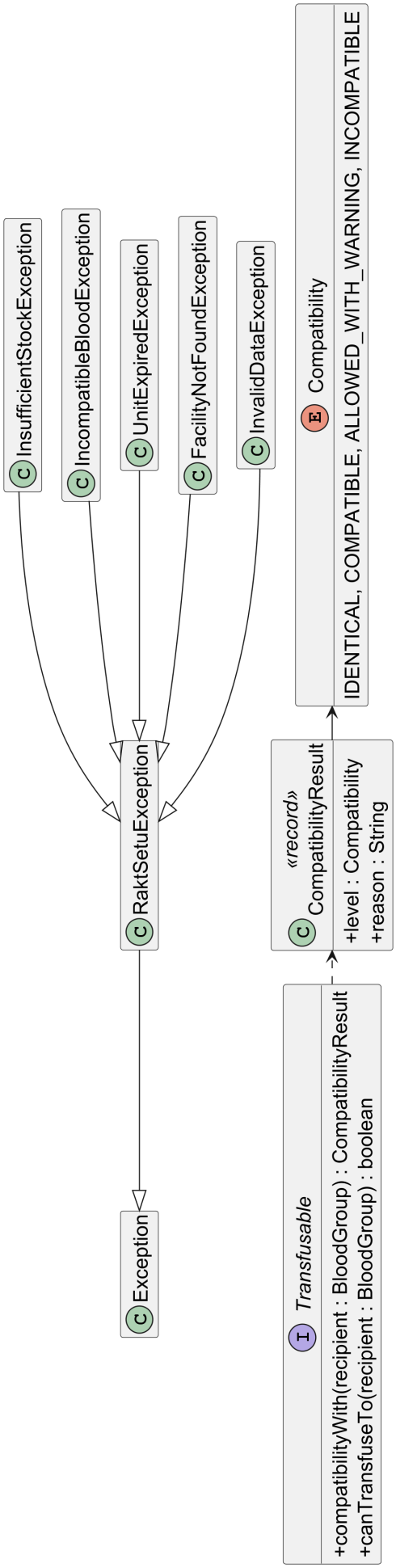


Figure 5.8: Engine classes: serving requests and approving transfers (turn the page)

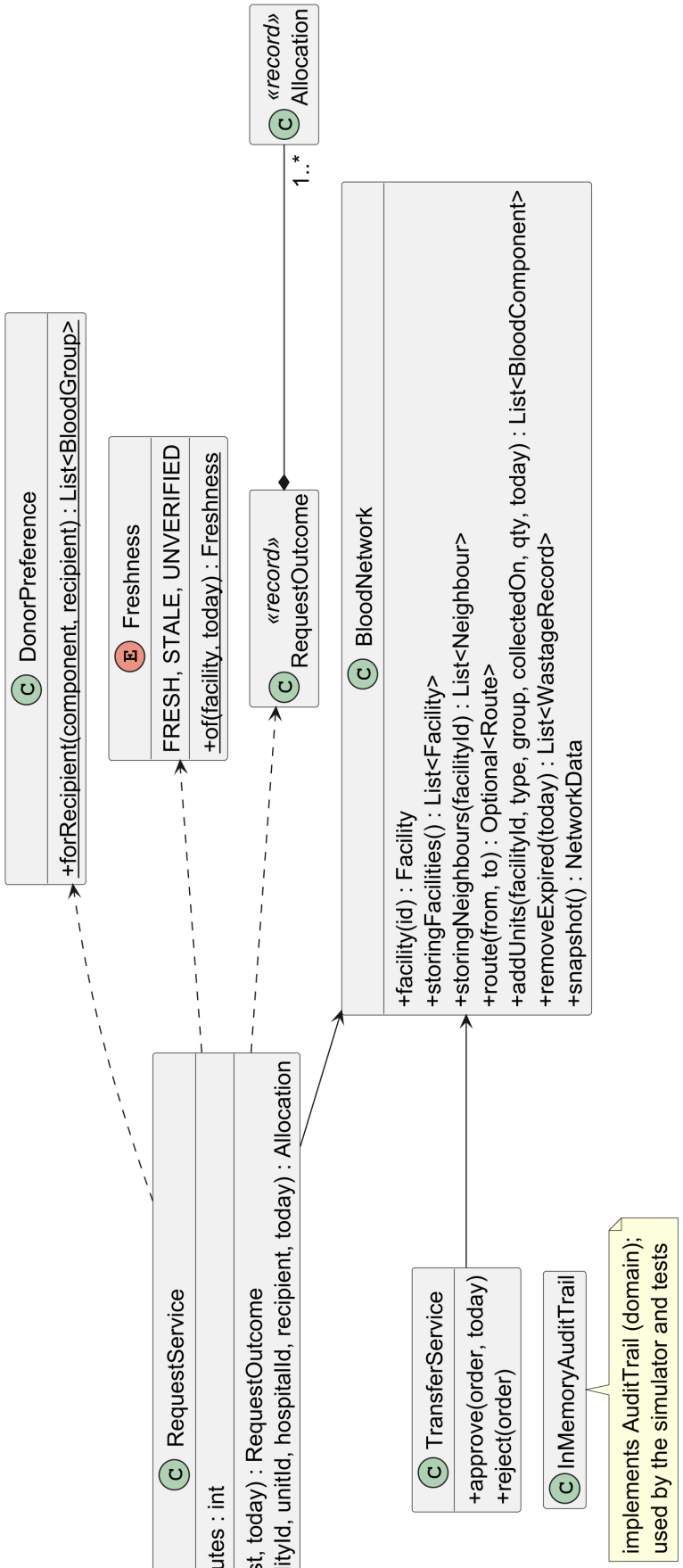


Figure 5.9: Engine classes: expiry risk and the redistribution strategies (Template Method) (turn the page)

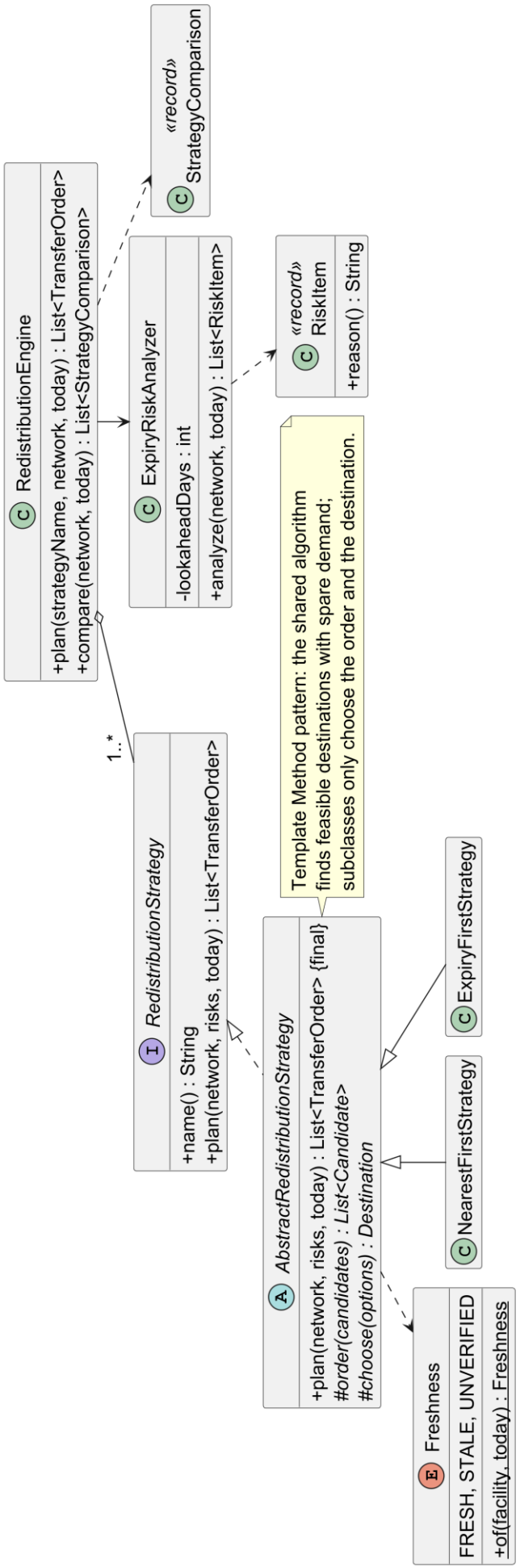


Figure 5.10: Demand forecasting: the forecasters (Strategy) and the Forecast objects they create (turn the page)

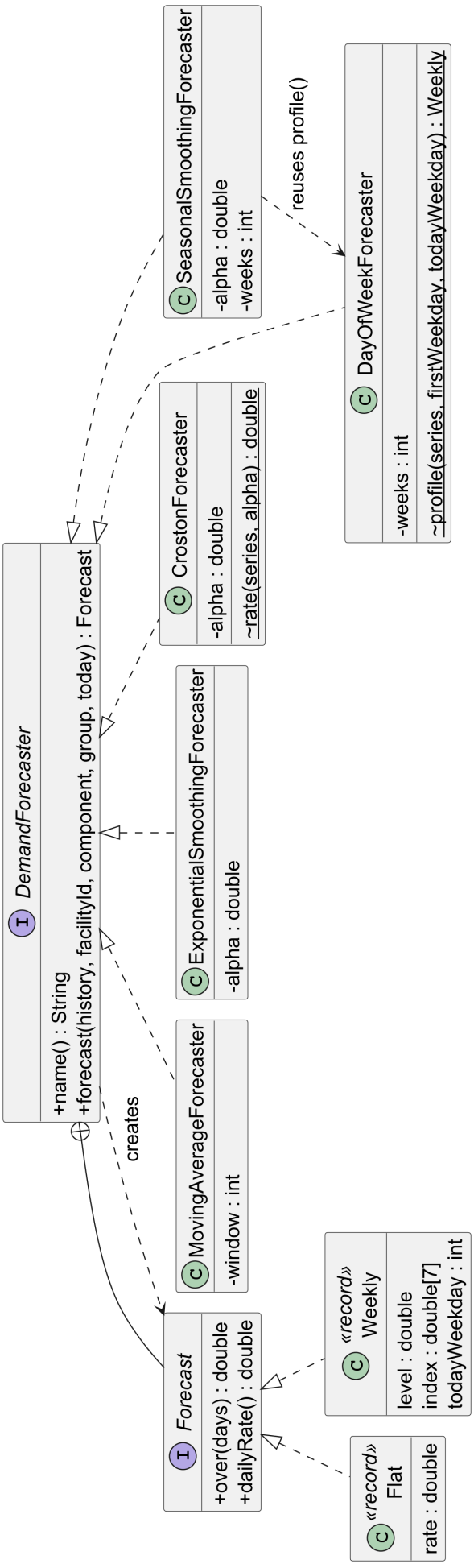


Figure 5.11: Demand forecasting: where the engine uses the forecasters (turn the page)

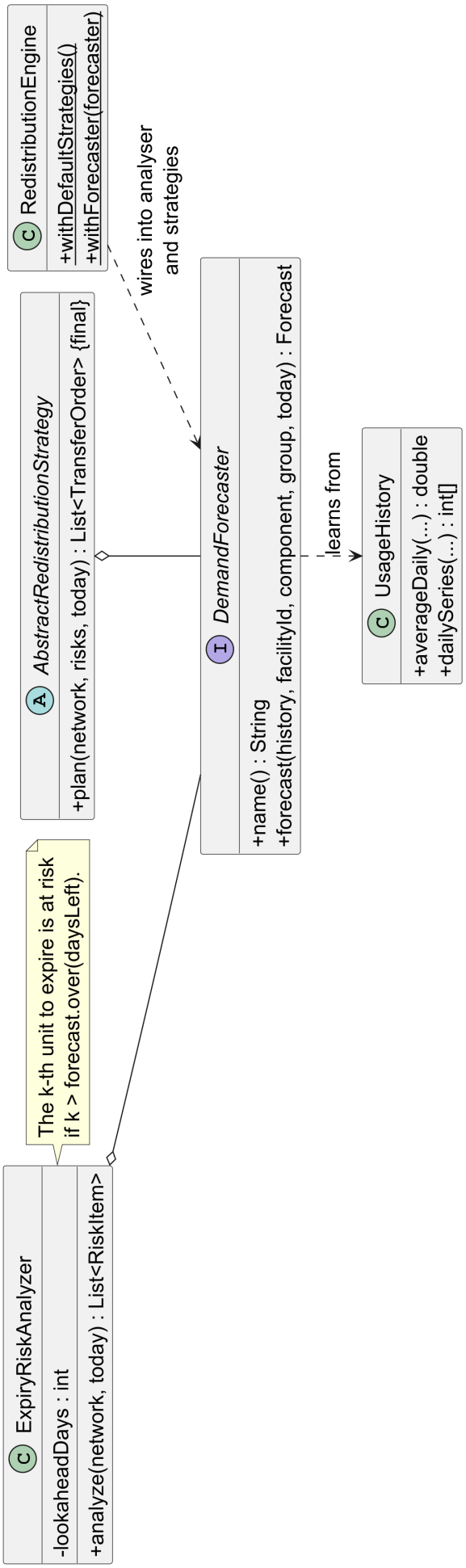


Figure 5.12: Server: sign-in lockout (Strategy) and error handling (turn the page)

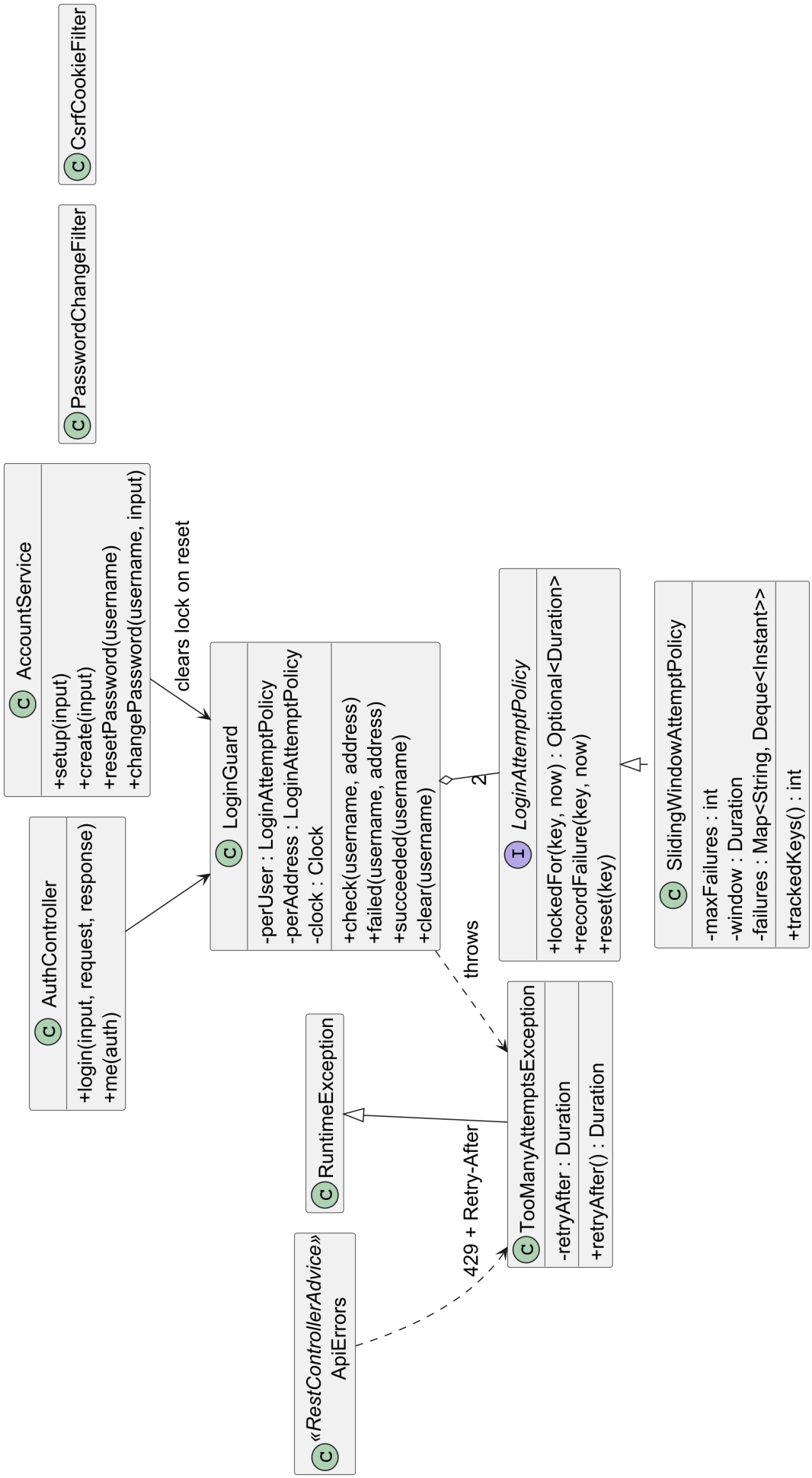


Figure 5.13: Server: health check, metrics and the readers-writer network lock (turn the page)

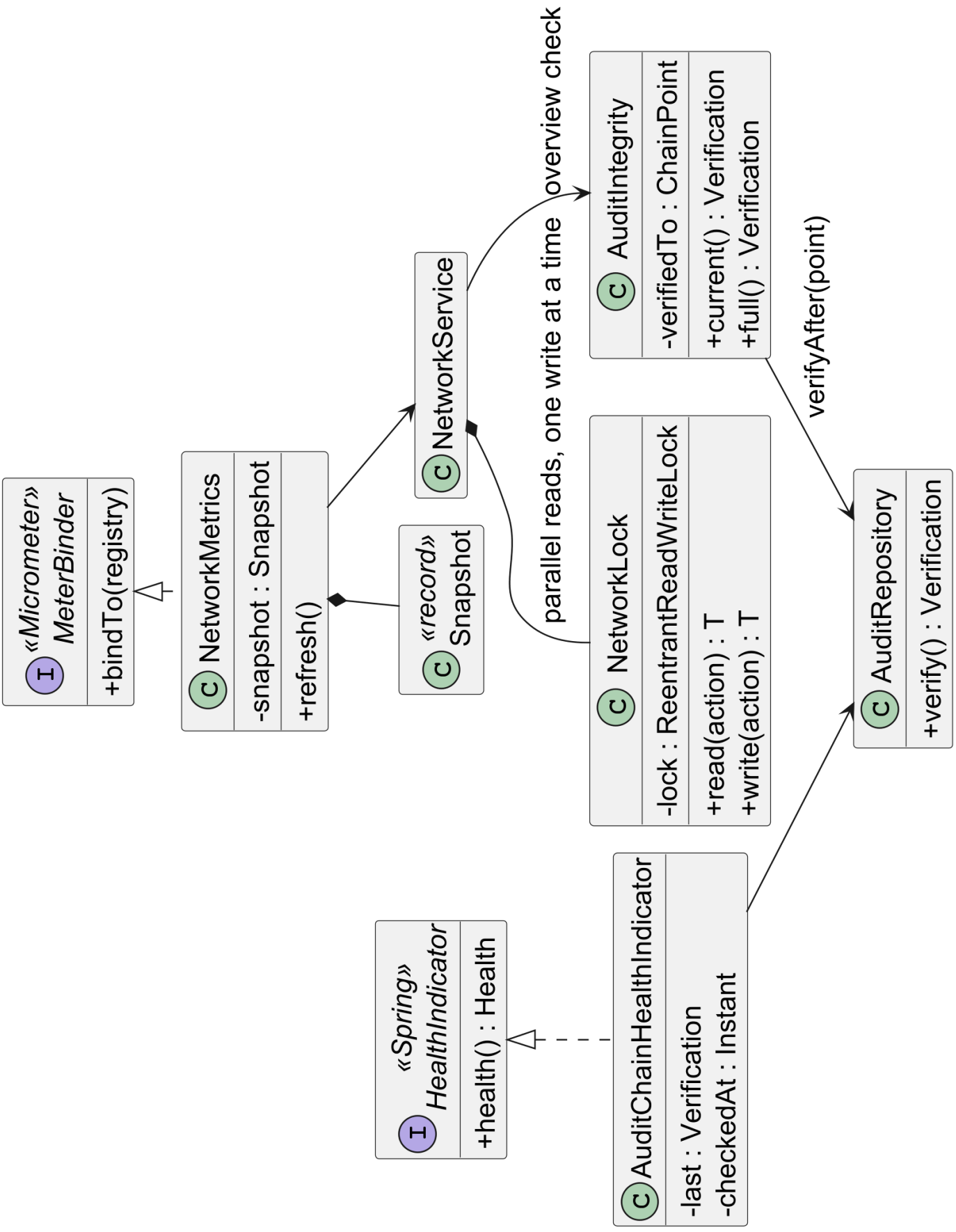


Figure 5.14: Urgent alerts: the alert channels (Strategy and Template Method) (turn the page)

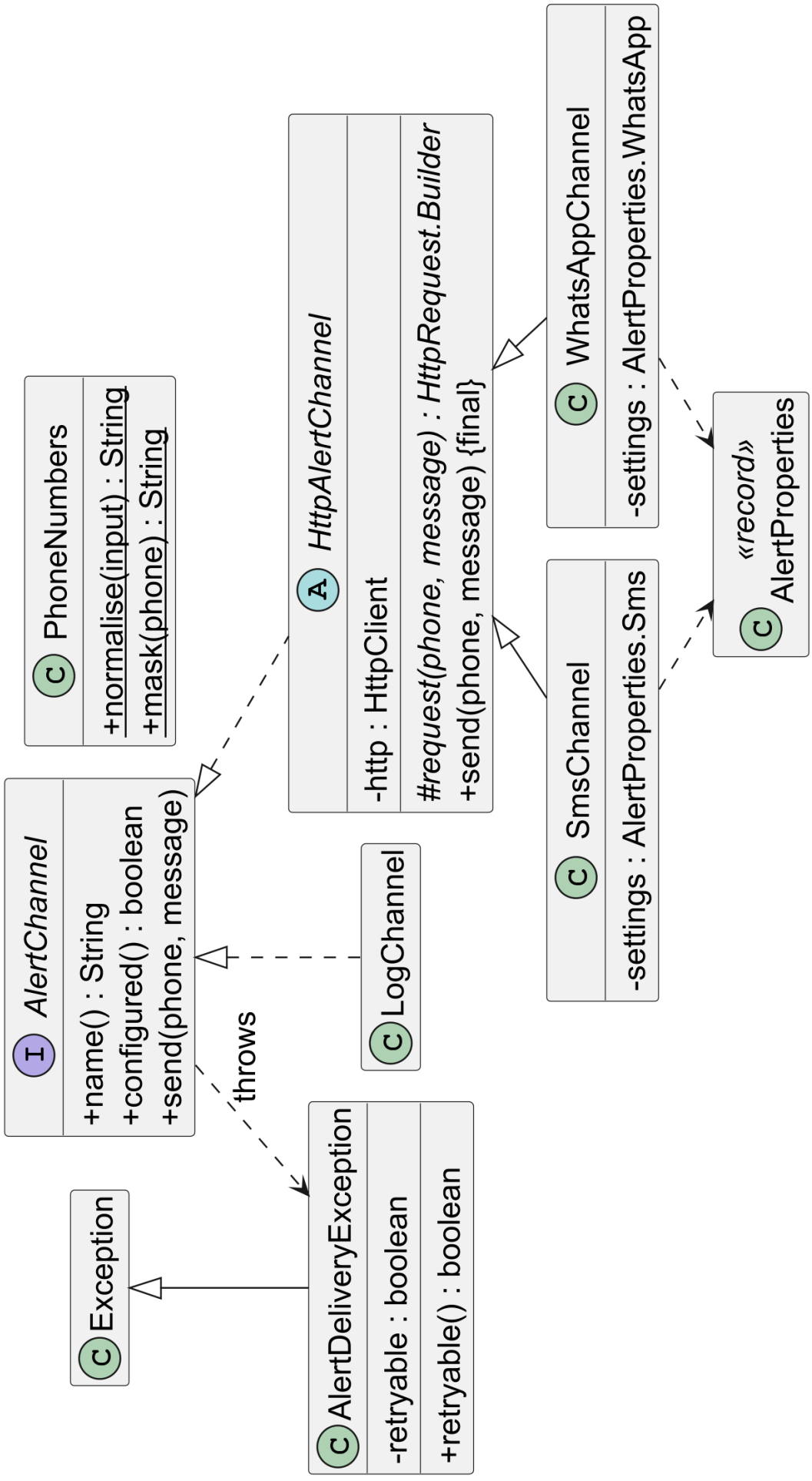


Figure 5.15: Urgent alerts: the transactional outbox and the dispatcher (turn the page)

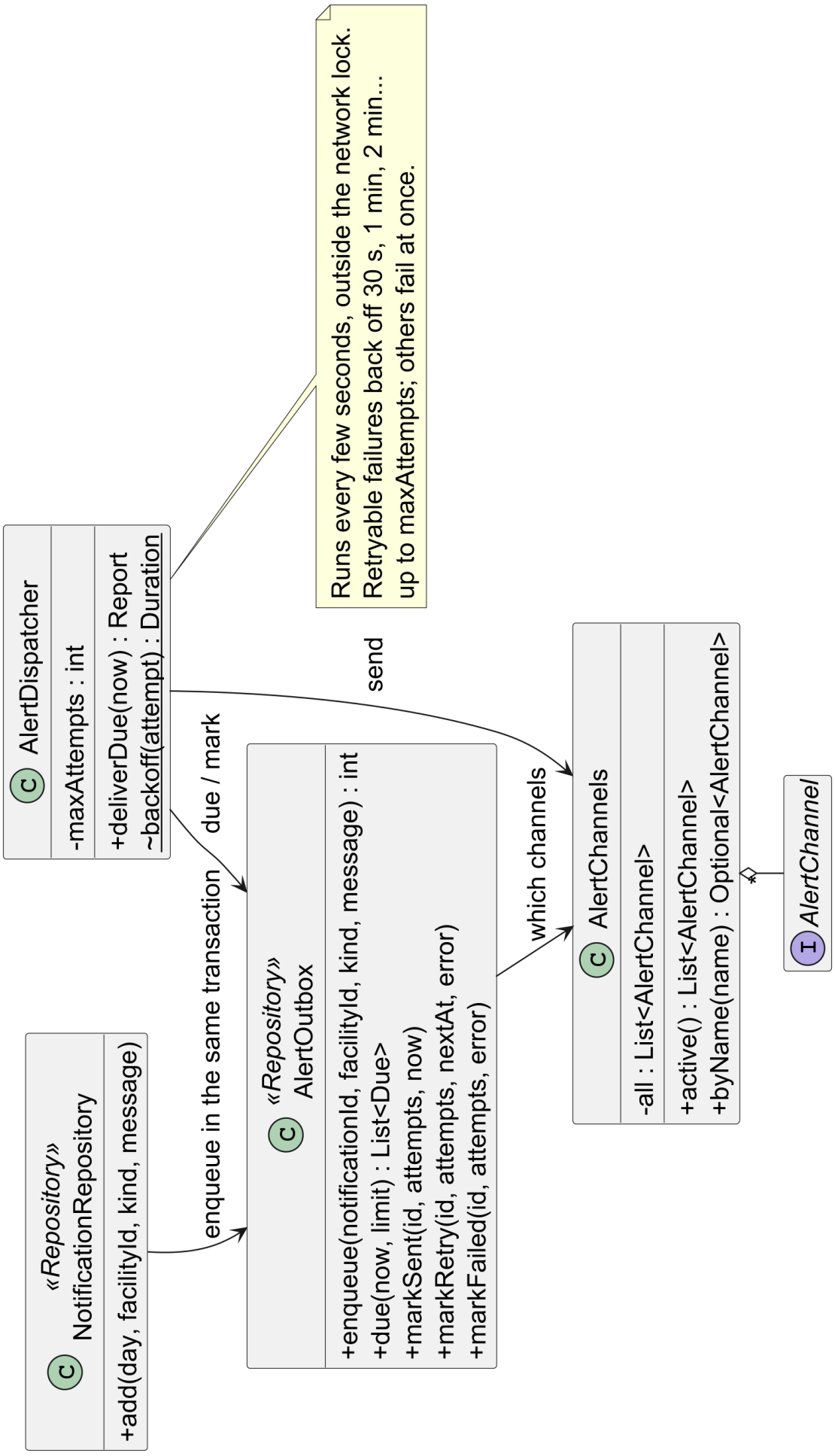
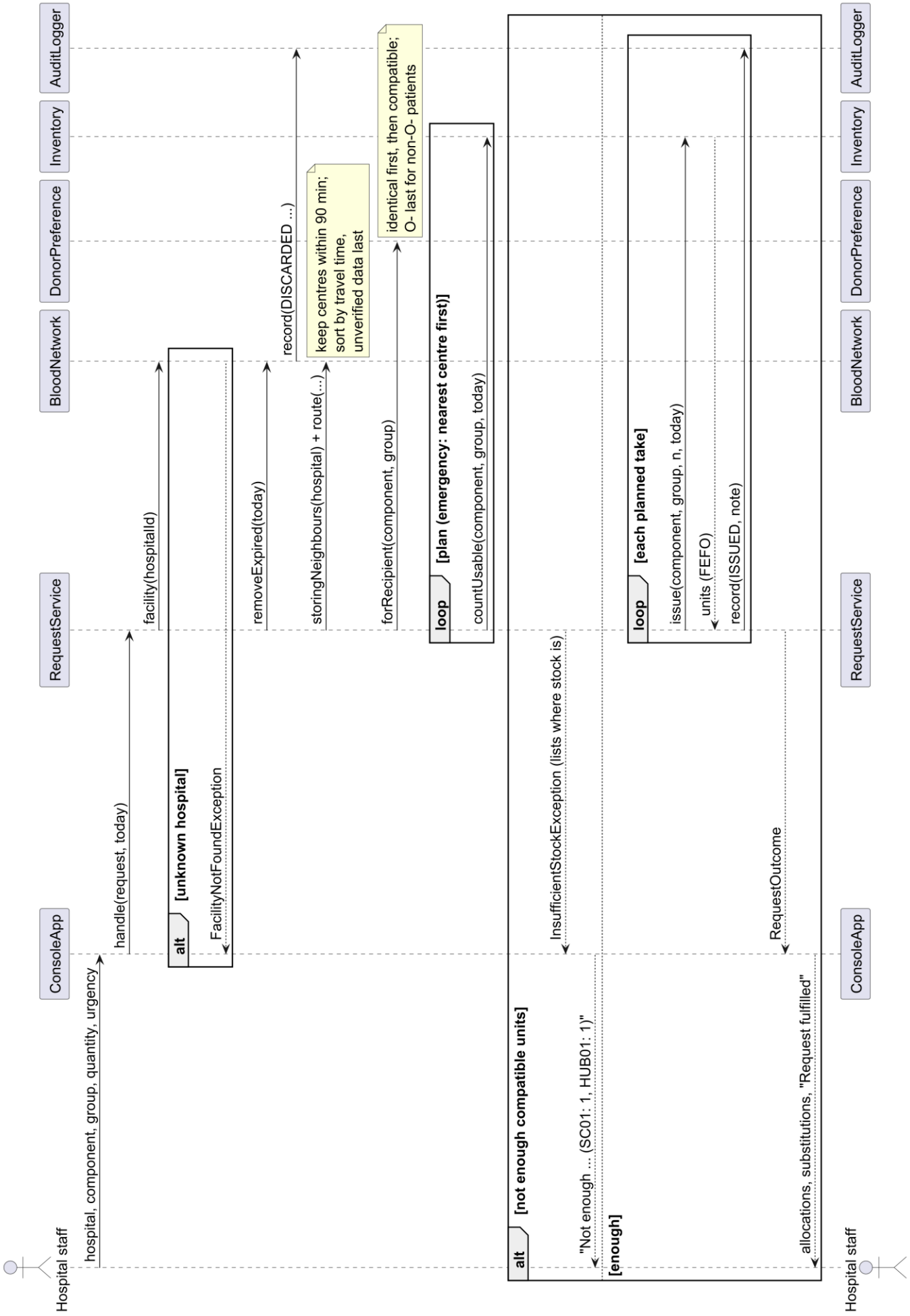


Figure 5.16: Sequence diagram: an emergency request from the hospital to issued units (turn the page)



6. Implementation

Six algorithms carry the system; each is short, deterministic and covered by tests.

6.1 Allocating a request (`RequestService`)

1. Remove expired units (cheap: each inventory remembers its earliest expiry and skips the scan when nothing can have expired).
2. Find centres with a route to the hospital within 90 minutes, from a neighbour index built once from the route table. Unverified centres go last.
3. Order donor groups: identical group first, then compatible ones; O- last for patients who are not O-.
4. Emergency: nearest centre first. Routine: exact group first across centres.
5. Take soonest-expiring units first. If the whole quantity cannot be met, issue nothing and record the request as unmet.

6.2 Expiry risk (`ExpiryRiskAnalyzer`)

Sort a centre's usable units of one component and group by expiry. The k-th unit is at risk if the demand forecast over its remaining days is less than k:

$$k > D(\text{daysLeft_k}), \quad \text{where } D_\text{movingAverage}(n) = (\text{average daily use}) \times n$$

Only units expiring within 7 days are reported, so long-lived plasma is not flagged months ahead.

6.3 Transfer planning (`AbstractRedistributionStrategy`)

For each at-risk unit, in the order the strategy chooses, a destination is feasible if it stores blood, its data is verified, the unit arrives with at least a day left, and it has spare demand:

$$\text{spare} = \text{floor}(D(\text{days on arrival})) - \text{usable stock} - \text{already sent}$$

Nearest-First sends each unit to the closest feasible centre; Expiry-First places the soonest-expiring units first. Transfers are same-group only and need staff approval.

6.4 Tamper-evident audit log (`AuditHashing`)

Each entry stores the hash of its own fields and the previous entry's hash, so editing, deleting or reordering any entry breaks every hash after it:

$$h_n = \text{SHA256}(n \,||\, \text{event_n} \,||\, h_(n-1)), \quad h_0 = \text{GENESIS}$$

The overview checks only entries added since its last check; the whole chain is re-checked every five minutes, on the audit page and by the health check.

6.5 Urgent alerts (transactional outbox)

When an urgent notification is saved, an outbox row per recipient and channel is written in the same transaction. A background dispatcher sends due rows every few seconds, outside the network lock.

Failures that may pass (HTTP 5xx, 429, network errors) are retried after 30 s, 1, 2 and 4 minutes, up to 5 tries; a wrong number or rejected key fails at once.

6.6 Concurrency (`NetworkLock`)

The in-memory network is shared by every user. A fair readers-writer lock lets any number of reads run together while a change has the network to itself. The daily roll-over runs before a read starts, because Java's lock cannot be upgraded from read to write.

7. Testing and quality

240 automated tests run on every change (234 in Java, 6 in the web front end), and the build fails if any module drops below 80% line coverage.

Module	Tests	What they check
raktsetu-server	80	API, sign-in and roles, CSRF, workflows, database write-through, lockout, health and metrics, concurrency, alerts end to end
raktsetu-engine	44	Allocation, compatibility order, expiry risk, strategies, transfers, forecasters (hand-checked values)
raktsetu-domain	57	Shelf lives, compatibility rules, inventory order and expiry, parsing
raktsetu-sim	22	Simulator, digital twin, demand patterns, the forecasting study end to end
raktsetu-persistence	17	CSV parsing and round trips, corrupted rows reported, audit file detects edited and deleted entries
raktsetu-cli	7	Every menu option, saving and reloading, bad input never crashes, tampered audit log detected
raktsetu-bench	7	Synthetic network generator, latency percentiles
web (Vitest)	6	Formatting helpers
Total	**240**	All pass (see Figure 11.9)

How quality is enforced

- **Coverage gate:** JaCoCo fails the build below 80% line coverage per module.
- **Continuous integration:** GitHub Actions builds and tests on Java 21 and 25, and runs the server tests a second time against PostgreSQL 17.
- **Equivalence checks:** each performance change and the forecasting refactor were checked by re-running the 30-day simulation for 5 seeds and 3 policies; the output was byte-identical before and after.
- **Concurrency test:** 16 threads raise, cancel and read at once; stock must end exactly as it started and the audit chain must be intact.
- **Fake providers:** SMS and WhatsApp requests are checked byte for byte against a local fake server, including retries and failures.

- **Browser checks:** the full first-run flow (setup, facilities, people, one-time password, a request from hospital to dispatch and receipt, an alert) was exercised in a real browser on an empty database.

8. Performance

Profiling found the bottlenecks, and fixing them made the engine 5 to 66 times faster at state scale and the server 4.4 times faster under load, with identical results.

8.1 Engine (JMH benchmarks)

Generated networks with 300 units per centre and two hospitals each; 1,000 centres means 3,000 facilities and 300,000 units.

Operation at 1,000 centres	Before	After	Speed-up	Main fix
Serve an emergency request	8.50 ms	0.13 ms	66x	Skip the expiry scan when nothing can have expired; search only nearby centres
Find units at risk of expiring	421 ms	76 ms	5.5x	Expiry date computed once; stock kept in a `TreeSet` instead of a `PriorityQueue`
Plan transfers for the network	31.6 s	0.67 s	47x	Neighbour index; per-plan lookup tables (planning time now grows linearly, not with the square of the network)

8.2 Server (HTTP load test)

Load test on 20 centres and 40 hospitals, 30 seconds per level, before and after tuning:

Users	Before: requests/s	Before: p99 (ms)	After: requests/s	After: p99 (ms)	Errors
1	83	54.6	133	21.5	0
10	116	195.9	413	53.4	0
50	100	1,767.1	428	193.0	0
100	86	2,071.7	432	414.3	0
200	96	3,159.8	426	736.3	0

The load test signs in hospital staff through the real API and runs a realistic mix: 85% reads, 15% raising and cancelling an emergency request. Three fixes produced the gain: the overview stopped re-hashing the whole audit log on every page (incremental verification), a readers-writer lock let reads run in parallel, and a list page went from 51 database queries to 2. At 200 users the slowest 1%

of responses fell from 3.16 s to 0.74 s. Throughput now levels off because each change holds the write lock while it saves to the database.

9. Research results

In a digital twin of a Pune district network, redistribution cut expired units by about 60%, while the choice of demand forecaster changed them by under 20 units in 120 days.

9.1 Set-up

The digital twin simulates 10 blood centres and 22 hospitals (synthetic facilities at real towns, OSRM road times) day by day. It keeps the true stock separate from what the planner sees through centres' reports. Each policy replays the same random days (common random numbers), so differences between policies are paired; 20 replications give 95% confidence intervals. Three demand scenarios were tested: flat, a weekday pattern, and a weekday pattern with a dengue-season platelet surge. Their magnitudes are illustrative.

9.2 Redistribution

Scenario	Expired units: no transfers to expiry-first	Unmet demand (share of units requested)
Flat demand	1,890 to 750 (-60%)	2.13% to 1.02%
Weekday pattern	1,913 to 757 (-60%)	2.09% to 1.02%
Weekday + dengue surge	1,509 to 342 (-77%)	9.81% to 8.53%

When supply is adequate, moving at-risk units to where they will be used cuts waste by about 60% and halves shortages. In the dengue scenario platelet demand doubles while supply does not, so most of the shortage remains.

9.3 Forecasting

Six forecasters were compared, including a 56-day moving average as a control for window length.

Forecast error relative to the 14-day moving average (lower is better), Pune pilot network, 20 replications:

Forecaster	3-day horizon (flat / weekly)	7-day horizon (flat / weekly)
Moving average, 14 days	1.00 / 1.00	1.00 / 1.00
Moving average, 56 days	0.94 / 0.94	0.88 / 0.87
Day of week	0.97 / 0.93	0.88 / 0.87
Seasonal smoothing	1.03 / 0.98	1.02 / 1.01
Exponential smoothing	1.02 / 1.02	1.03 / 1.03
Croston / SBA	0.97 / 0.98	0.93 / 0.93

- A longer history made forecasts up to 13% more accurate; modelling the weekday added about 1 point beyond that, and Croston/SBA did not help rare groups.

- More accurate forecasts did not give better plans: wastage tracked how many units were moved (correlation -0.93 to -0.96), not forecast error. Forecasts for this decision should therefore be judged by the wastage and shortages they lead to.

Full method, tables and threats to validity are in `docs/research/forecasting/`.

9.4 Supporting research

Published work supports the approach. Hajjaj et al. (Transfusion, 2024) studied the redistribution of near-expiry blood products from hospitals with low usage to hospitals more likely to use them, across 130 hospital transfusion services in Ontario, Canada, between 2012 and 2020. Redistributing 15,499 products prevented the outdating of about CAD 17.6 million of blood products at a cost of about CAD 45,000 for shipping and staffing. The study also evaluated transport failures: 97 shipments (816 products) were discarded, most often because of courier delays [9]. RaktSetu applies the same idea across a network, chooses where each unit should go, and records every move.

10. Application snapshots

These screenshots are from the running web application. Figures 10.1 to 10.3 are from the live deployment; the rest are from a local copy seeded with seven days of sample activity, seen as the network administrator.

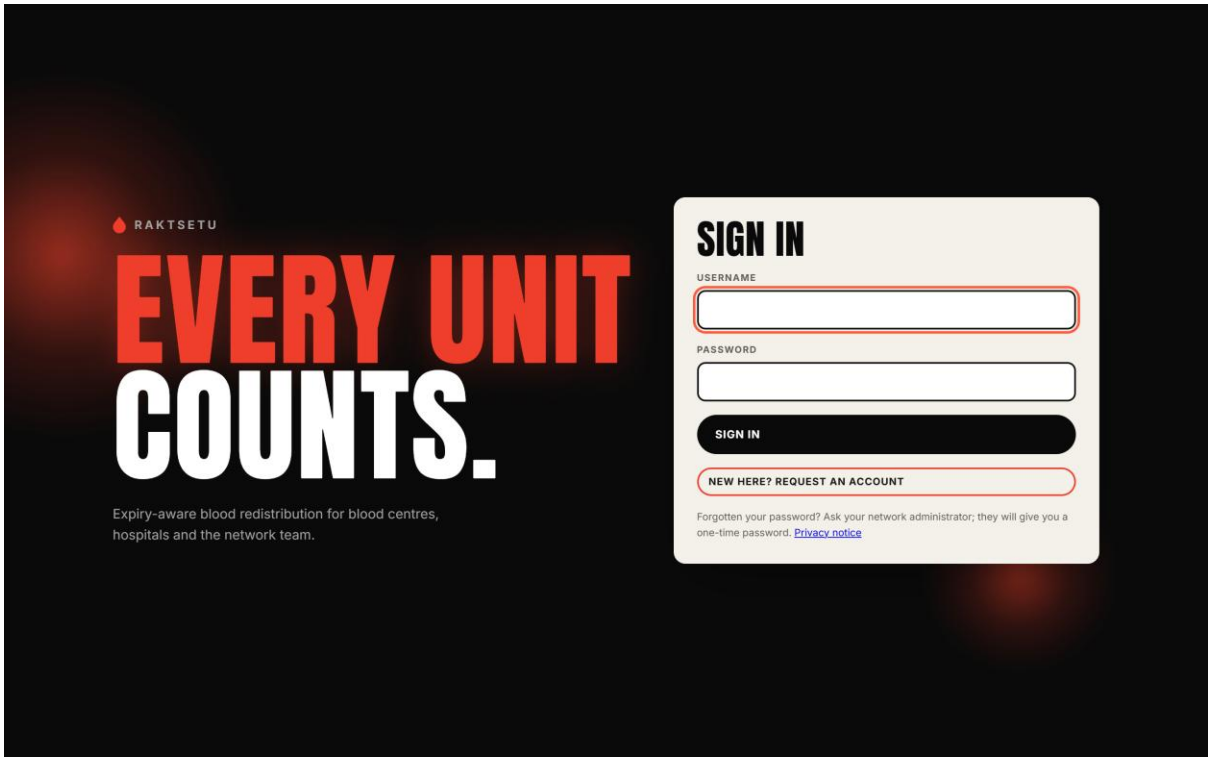


Figure 10.1: Sign-in page of the live site (raktsetu-ubvx.onrender.com)

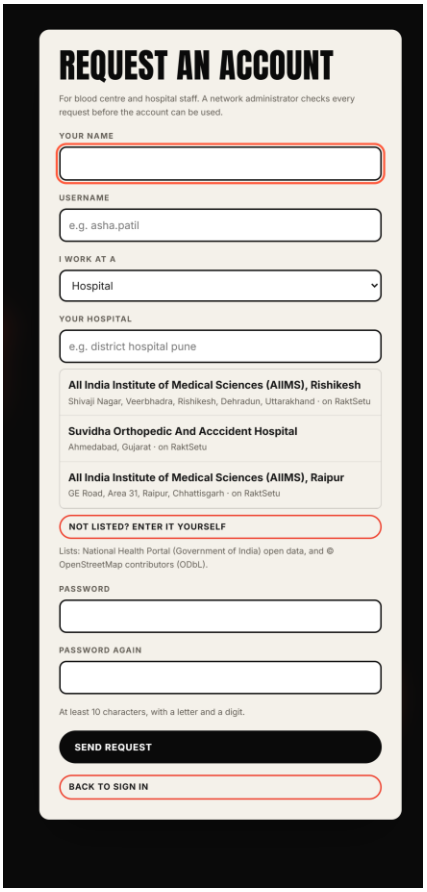


Figure 10.2: Request an account: search the national facility directory, or enter a facility that is not listed

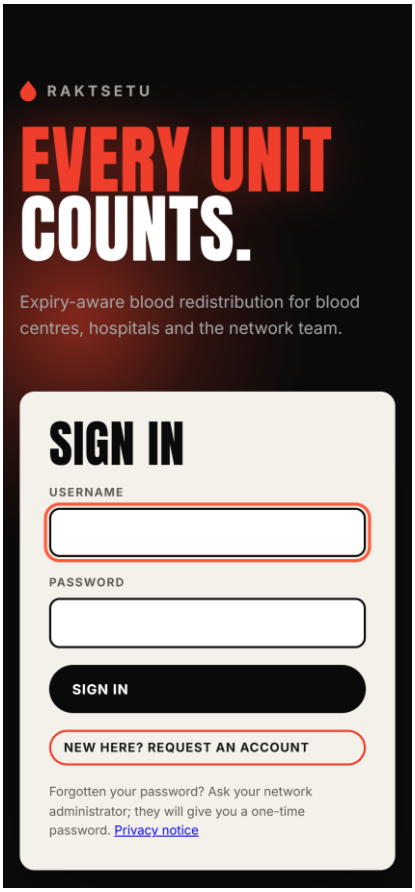


Figure 10.3: The live site on a phone

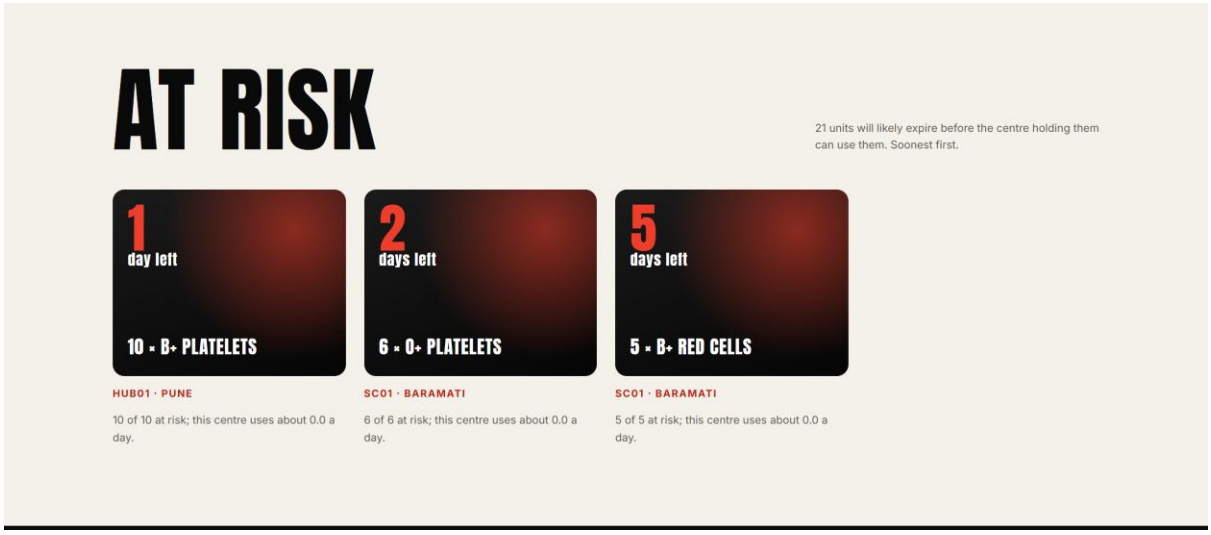


Figure 10.4: At-risk units, soonest expiry first

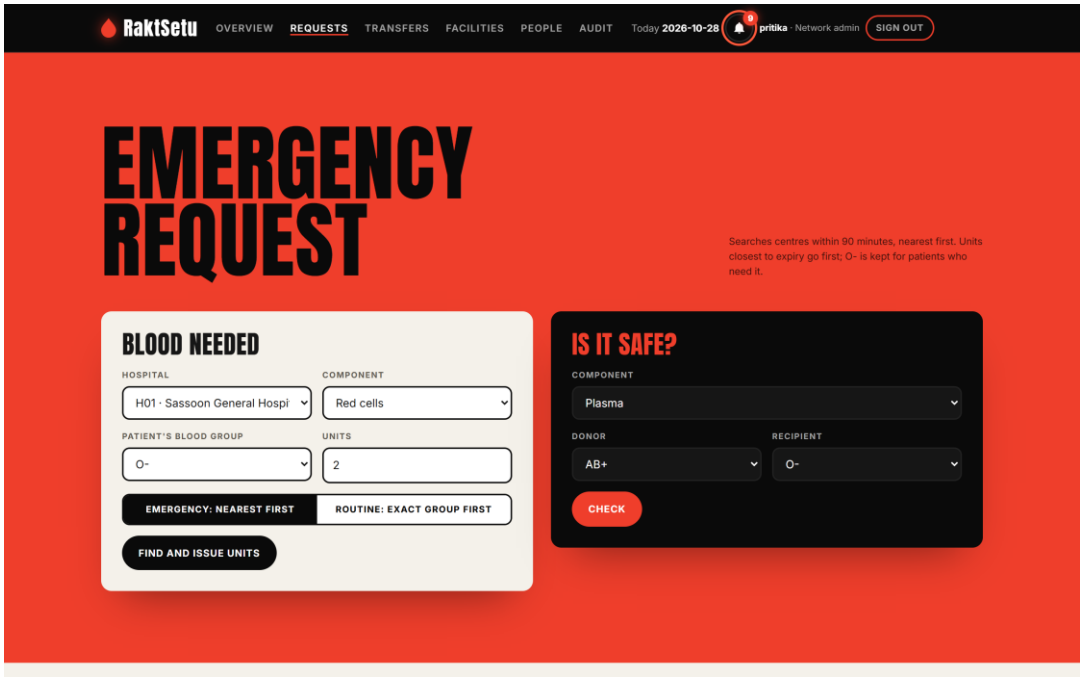


Figure 10.5: Requests: emergency request form and the "Is it safe?" compatibility check

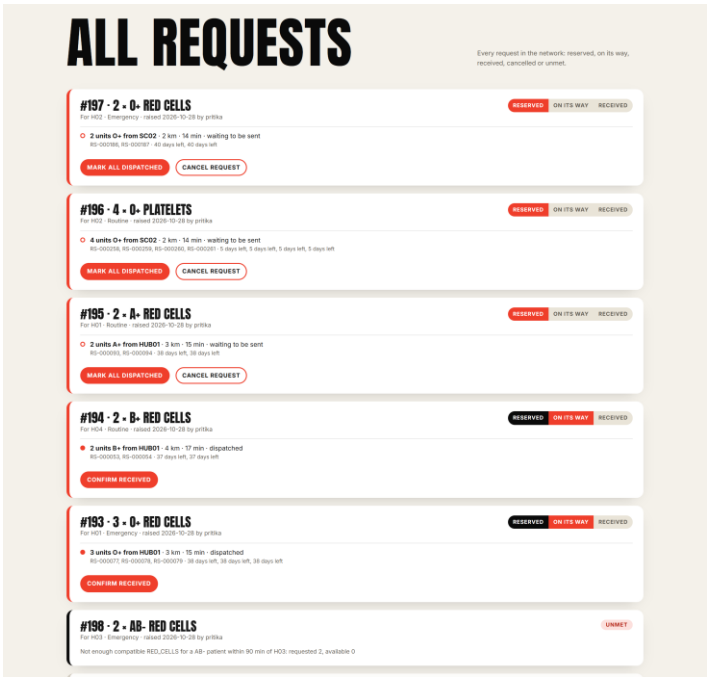


Figure 10.6: All requests, each tracked from Reserved to On its way to Received; unmet ones are flagged

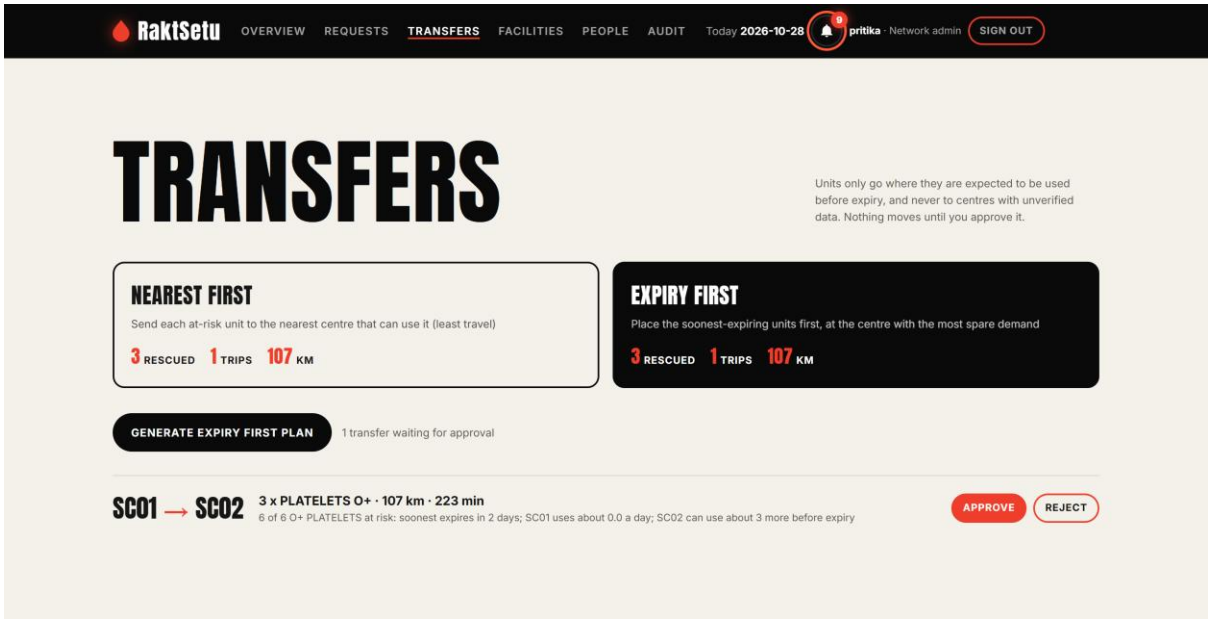


Figure 10.7: Transfers: nearest-first and expiry-first compared, and a plan waiting for approval

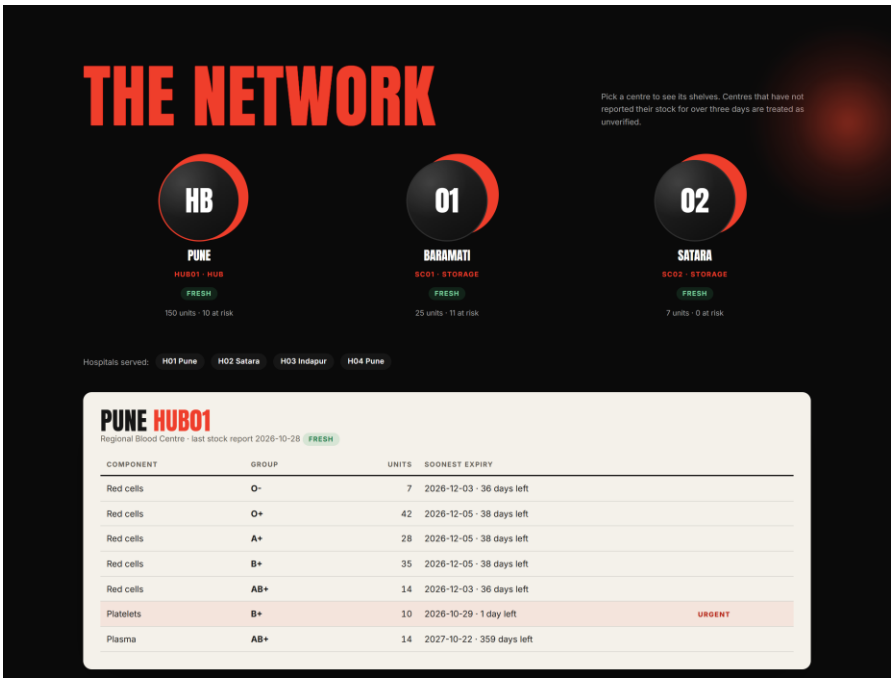


Figure 10.8: The network: each centre's stock by component and blood group

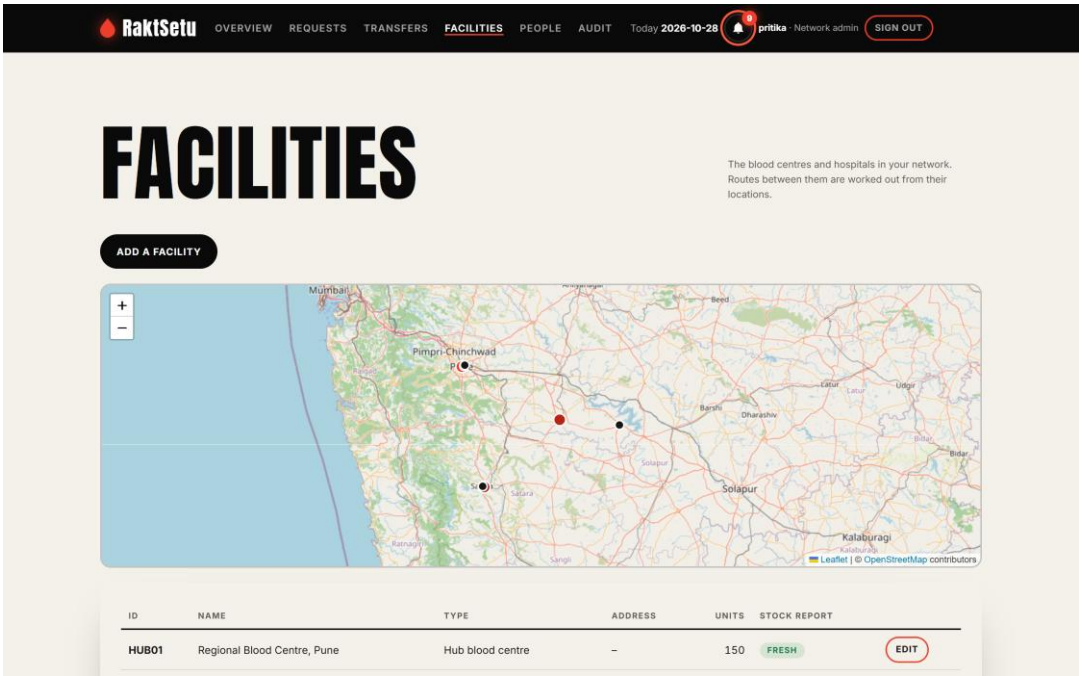


Figure 10.9: Facilities on the map

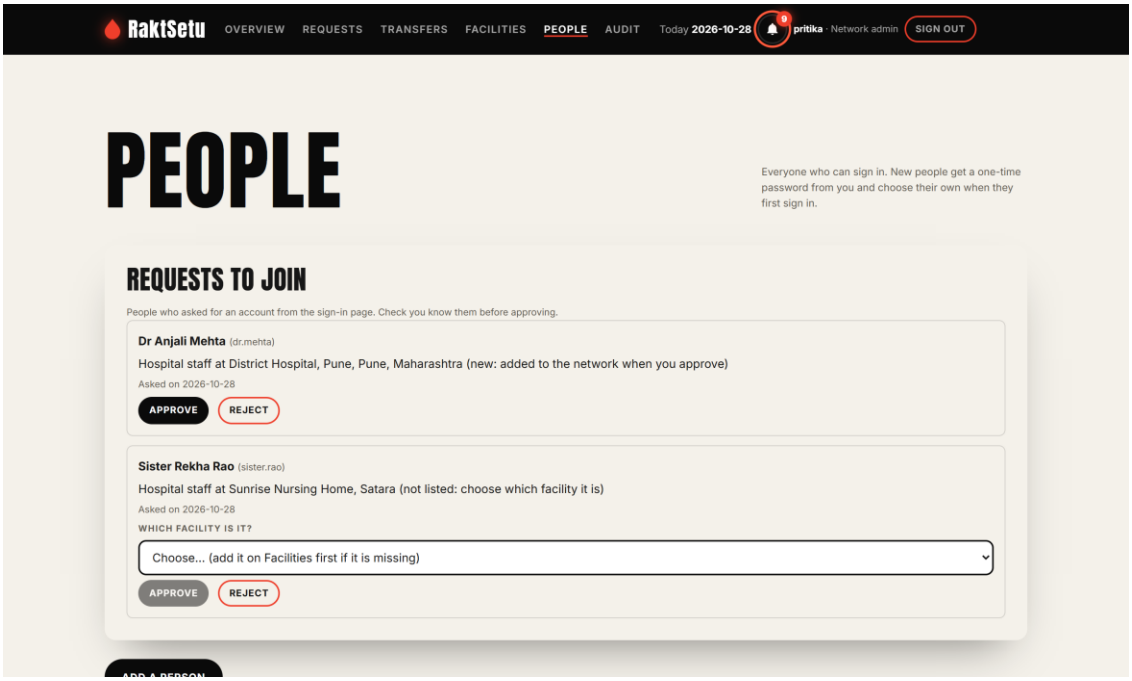


Figure 10.10: People: join requests approved or rejected by the administrator

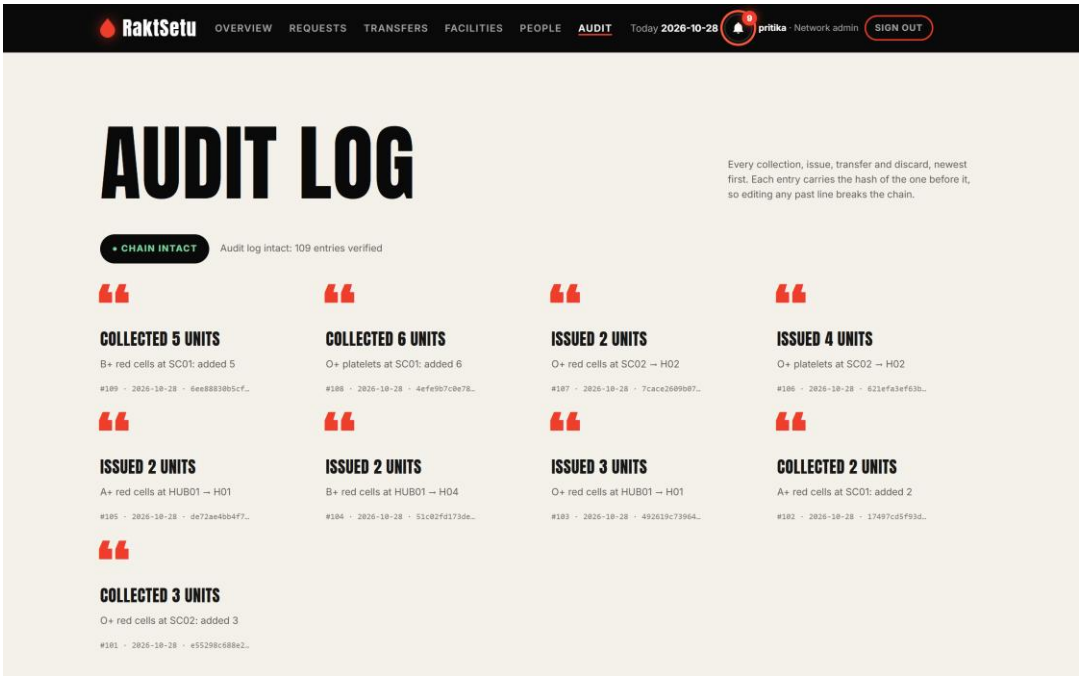


Figure 10.11: Audit log, with the hash-chain check

11. Program output

Output from the console application (run with a fixed date, 1 October 2026, and seed 42) and from the automated test run.

RaktSetu console

```
$ java -jar raktsetu-cli/target/raktsetu.jar --today 2026-10-01
RaktSetu - expiry-aware blood redistribution network
Created a sample network in raktsetu-data (seed 42, dated relative to 2026-10-01).
Today is 2026-10-01. 11 facilities, 329 units in stock.
3 expired units were removed from stock and recorded as wastage (see option 12).

===== RAKTSETU  (today: 2026-10-01, pending transfers: 0) =====
 1. View centre stock
 2. View network-wide stock
 3. Add blood units
 4. Raise a blood request
 5. Check compatibility
 6. Run the expiry-risk check
 7. Generate a redistribution plan
 8. Compare strategies
 9. Approve or reject transfers
10. View audit log and verify hash chain
11. Run the simulation
12. View the wastage summary
13. Export reports (CSV)
14. Data-freshness report
15. Set the date ("today")
16. Save and exit
Choose an option (1-16):
```

Figure 11.1: Console application: start-up and main menu

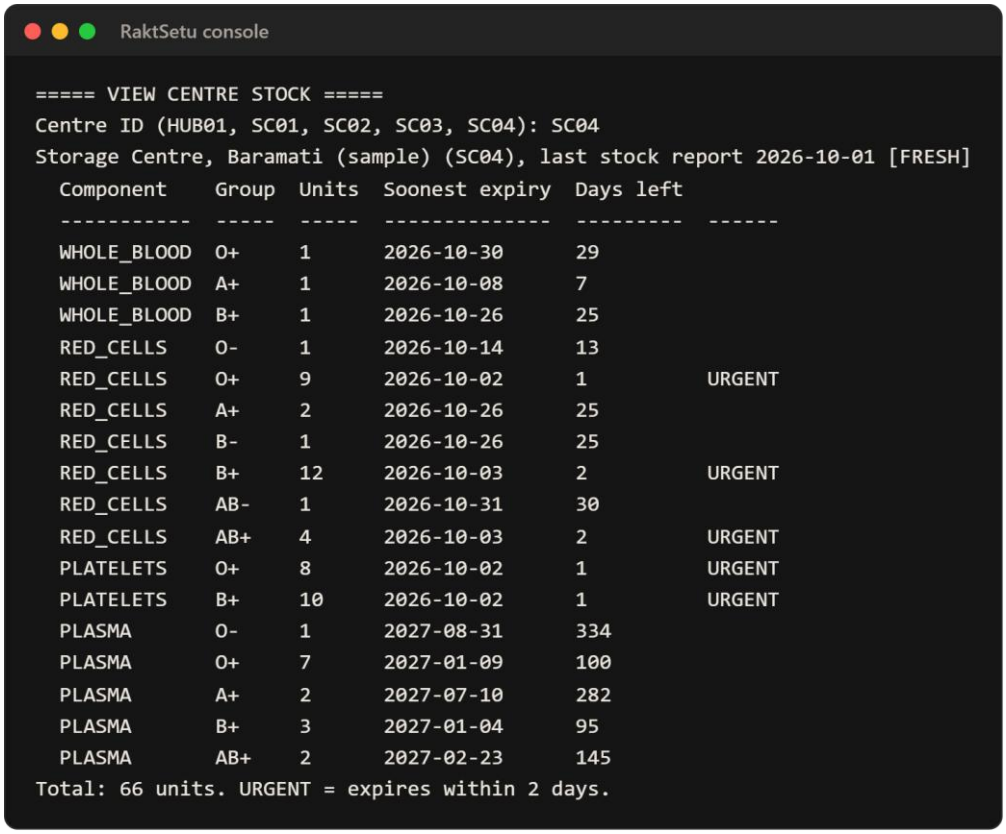


Figure 11.2: Option 1: stock at the Baramati storage centre, with units that expire within two days marked URGENT

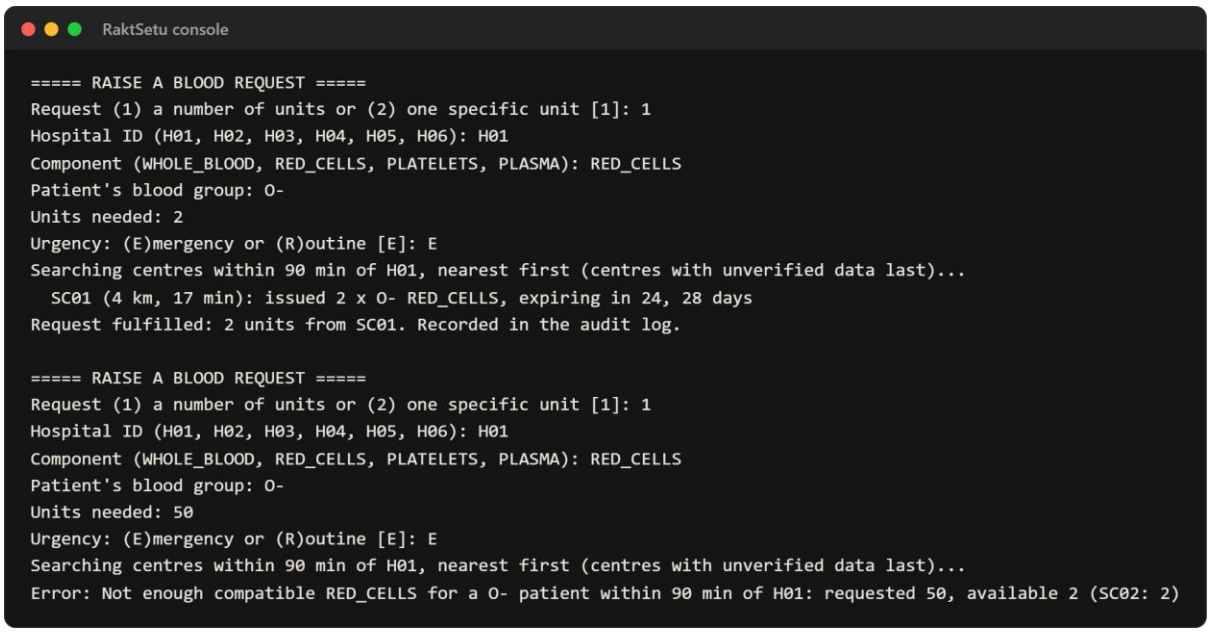


Figure 11.3: Option 4: an emergency request served nearest-first, then a request refused with a clear message (exception handling)

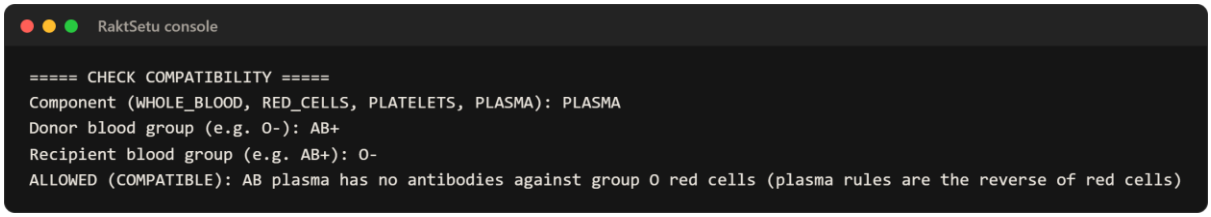


Figure 11.4: Option 5: compatibility check; plasma rules are the reverse of red cells

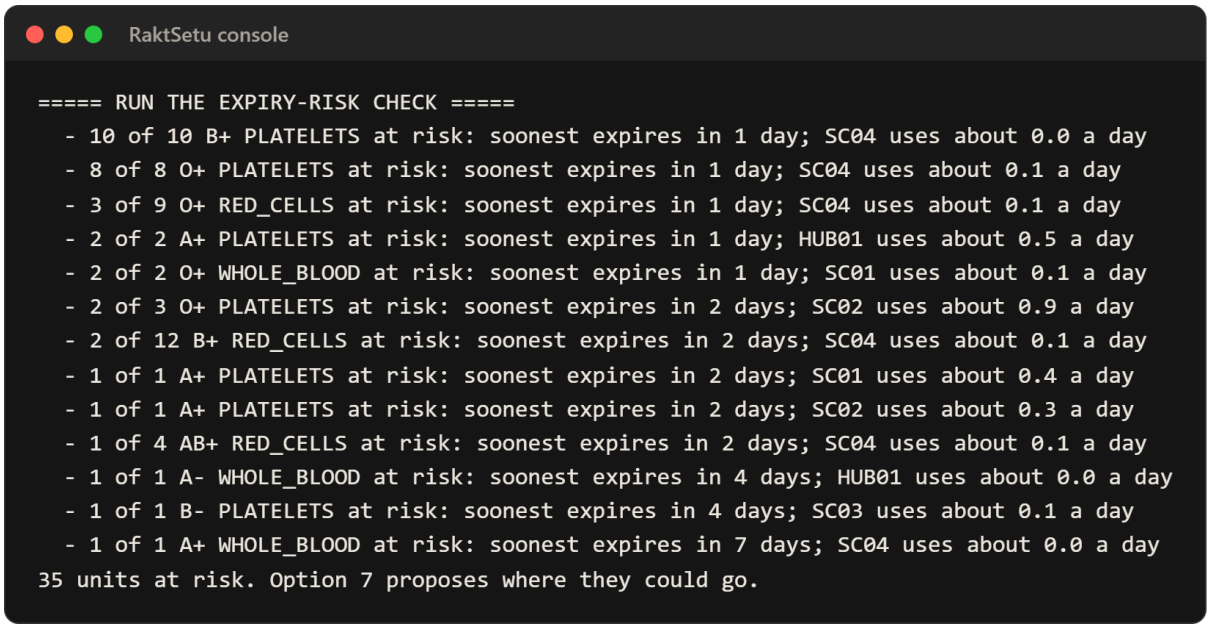


Figure 11.5: Option 6: the expiry-risk check

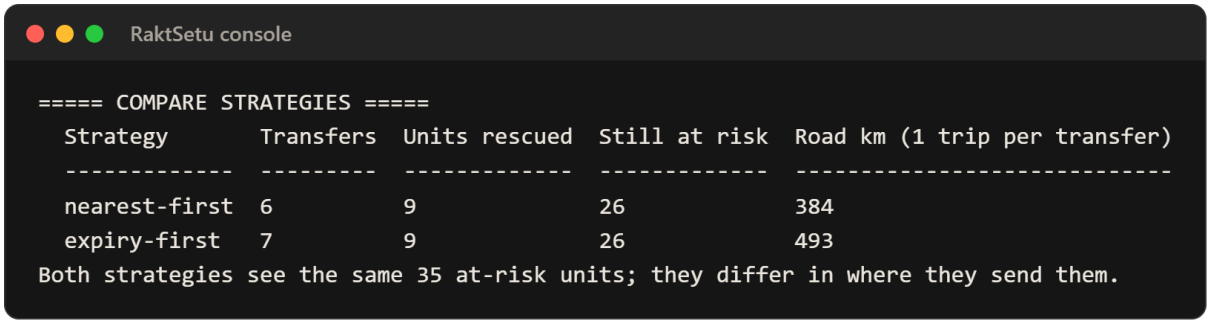


Figure 11.6: Option 8: both redistribution strategies on the same data

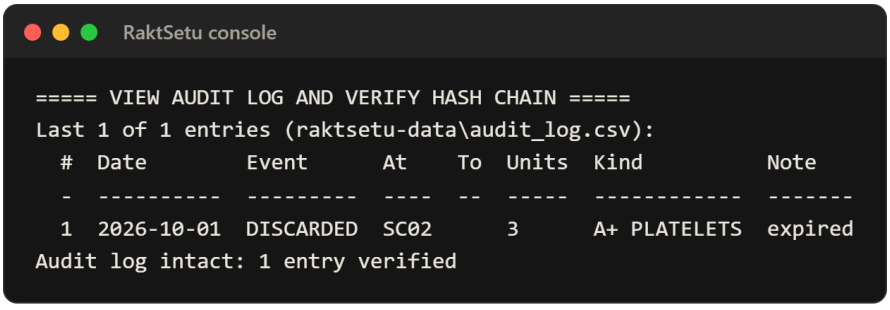


Figure 11.7: Option 10: the audit log and its SHA-256 hash-chain check

RaktSetu console

```
===== RUN THE SIMULATION =====
Days to simulate [30]: 30
Simulating 30 days from 2026-10-01 on a fresh copy of the sample network (seed 42).
Every policy sees the same supply and demand; your data is not changed...
  Measure             no redistribution  nearest-first  expiry-first
  ------------------  -----------------  -------------  ------------
  Units expired       105                68             68
    platelets         91                 59             59
    red cells         10                 5              5
    whole blood       4                  4              4
    plasma            0                  0              0
  Requests fulfilled  1041               1052           1052
  Requests unmet      43                 32             32
  Units transferred   0                  174            152
Wastage reduced by 35.2% with nearest-first.
Wastage reduced by 35.2% with expiry-first.
Saved to raktsetu-data\reports\simulation.csv
```

Figure 11.8: Option 11: the 30-day simulation; redistribution cuts expired units from 105 to 68

RaktSetu console

```
$ ./mvnw test
[INFO] Tests run: 57, Failures: 0, Errors: 0, Skipped: 0     raktsetu-domain
[INFO] Tests run: 44, Failures: 0, Errors: 0, Skipped: 0     raktsetu-engine
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0     raktsetu-persistence
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0     raktsetu-sim
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0      raktsetu-cli
[INFO] Tests run: 80, Failures: 0, Errors: 0, Skipped: 0     raktsetu-server
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0      raktsetu-bench
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for RaktSetu 0.1.0-SNAPSHOT:
[INFO] RaktSetu ........................................... SUCCESS [  1.266 s]
[INFO] raktsetu-domain .................................... SUCCESS [  5.753 s]
[INFO] raktsetu-engine .................................... SUCCESS [  6.269 s]
[INFO] raktsetu-persistence ............................... SUCCESS [  5.045 s]
[INFO] raktsetu-sim ....................................... SUCCESS [ 12.982 s]
[INFO] raktsetu-cli ....................................... SUCCESS [  5.921 s]
[INFO] raktsetu-server .................................... SUCCESS [02:28 min]
[INFO] raktsetu-bench ..................................... SUCCESS [  6.247 s]
[INFO] BUILD SUCCESS
[INFO] Total time:  03:12 min
```

Figure 11.9: Build and test run: all 234 Java tests pass

12. User guide

The live site needs only a browser: https://raktsetu-ubvx.onrender.com. To run it locally, the only requirement is Java 21 or newer; one command builds everything, and one starts the web app at http://localhost:8080.

12.1 Build and run

```
./mvnw package -DskipTests
java -jar raktsetu-server/target/raktsetu-server.jar
```

On Windows, double-click `run-web.bat` instead. Maven downloads its own Node.js to build the web front end, so nothing else needs installing. Options: `--raktsetu.data-dir=<folder>` chooses where the database lives; `--server.port=<port>` changes the port.

The console application (the course V1 deliverable) runs with `java -jar raktsetu-cli/target/raktsetu.jar` and offers 16 menu options on a sample network.

12.2 First use

1. **Setup:** the first visit asks for the first network administrator's name, username and password (at least 10 characters, with a letter and a digit).
2. **Facilities:** add the hub, storage centres and hospitals by clicking each location on the map. Road routes are worked out automatically and can be corrected by hand.
3. **People:** create an account for each member of staff. RaktSetu shows a one-time password once; the person chooses their own at first sign-in.
4. **Stock:** centre staff log collections (with the donation number on the bag, or numbered automatically) and report stock daily.
5. **Requests:** hospital staff raise emergency or routine requests; the reserved centres are notified, dispatch, and the hospital confirms receipt.
6. **Transfers:** the administrator generates a plan, then approves or rejects each proposed transfer.

12.3 Optional settings

- **Urgent alerts:** each person can add a mobile number on their Account page. To send real SMS or WhatsApp messages, set the Twilio or Meta Cloud API keys as environment variables (listed in the README).
- **PostgreSQL:** `docker compose up --build`, or start the JAR with `--spring.profiles.active=postgres`.
- **API documentation:** http://localhost:8080/api-docs; health check: http://localhost:8080/actuator/health.

13. Limitations, future work and conclusion

RaktSetu works end to end and its design choices are measured, but its outcome figures come from simulation and still need real data.

13.1 Limitations

- **Synthetic data:** the Pune network places synthetic facilities at real towns; demand, supply and the weekday and dengue patterns are illustrative, not calibrated.
- **Censored usage:** usage is recorded as units issued, so demand that went unmet during a stock-out is invisible to forecasting.
- **One server:** the network lives in one server's memory behind one lock; throughput levels off at about 430 requests per second on a laptop because each change holds the write lock while it saves.
- **Decision support only:** every transfer needs human approval, and compatibility rules must be confirmed against current NACO/NBTC guidelines before any clinical use.
- **Alerts need an account:** real SMS or WhatsApp messages need a Twilio or Meta account; without one, alerts go to the server log.

13.2 Future work

1. Calibrate demand, supply and wastage against real usage data, for example from e-Raktkosh or a partner hospital.
2. Repeat the digital-twin study at 50-70% reporting compliance, and forecast from requests rather than issues to remove censoring.
3. Save changes outside the write lock and partition the network by region to scale beyond one server.
4. Read ISBT 128 bag barcodes and exchange data in e-Raktkosh's format.
5. Have a transfusion doctor sign off the clinical rules, and connect a paid SMS provider for alerts in India.

13.3 Conclusion

RaktSetu shows that an object-oriented design can carry a real decision problem from domain model to working product. On the simulated network, moving at-risk units to where they will be used cut expired units by about 60% and halved shortages. The same model runs in a console application, a multi-user web app and a research simulator, all held to one test suite.

The forecasting study adds a finding worth keeping: a more accurate demand forecast did not produce a better plan, so forecasts for this decision should be judged by the wastage and shortages that result, not by forecast error.

References

1. J. D. Croston, "Forecasting and stock control for intermittent demands," *Operational Research Quarterly*, vol. 23, no. 3, pp. 289-303, 1972.

2. A. A. Syntetos and J. E. Boylan, "The accuracy of intermittent demand estimates," *International Journal of Forecasting*, vol. 21, no. 2, pp. 303-314, 2005.

3. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

4. Ministry of Health and Family Welfare, Government of India, e-Raktkosh: Centralized Blood Center Management System. [Online]. Available: https://eraktkosh.mohfw.gov.in/ (accessed 1 October 2026).

5. Project OSRM, Open Source Routing Machine, used for road distances and travel times. [Online]. Available: https://project-osrm.org/ (accessed 1 October 2026).

6. OpenJDK, Java Microbenchmark Harness (JMH) 1.37. [Online]. Available: https://github.com/openjdk/jmh (accessed 1 October 2026).

7. Spring Boot 4.1 and Spring Security 7 reference documentation. [Online]. Available: https://docs.spring.io/spring-boot/ and https://docs.spring.io/spring-security/reference/ (accessed 1 October 2026).

8. Rajya Sabha, Unstarred Question No. 1882, answered 4 August 2026, citing e-Raktkosh data on blood collection and discards. Reported in Factly, "Blood collection rose 24% in India, while discards more than doubled." [Online]. Available: https://factly.in/blood-collection-rose-24-in-india-while-discards-more-than-doubled/ (accessed 1 October 2026).

9. O. I. Hajjaj, D. Modi, T. Cameron, R. Barty, W. Owens, N. Heddle, L. Zhang, T. Thompson and J. Callum, "Reducing blood product wastage through the inter-hospital redistribution of near-outdate inventory," *Transfusion*, vol. 64, no. 7, pp. 1207-1216, 2024, doi: 10.1111/trf.17876. PubMed: https://pubmed.ncbi.nlm.nih.gov/38752381/.

10. PlantUML, open-source UML diagram tool. [Online]. Available: https://plantuml.com/ (accessed 29 October 2026).

11. Ministry of Health and Family Welfare, National Health Portal: hospital and blood bank directories (open data), accessed through Esri Living Atlas. [Online]. Available: https://livingatlas.arcgis.com/ (accessed 20 October 2026).

12. OpenStreetMap contributors, hospital data via the Overpass API, licensed under the Open Database License (ODbL). [Online]. Available: https://www.openstreetmap.org/copyright (accessed 20 October 2026).

13. React 19 and TypeScript documentation. [Online]. Available: https://react.dev/ and https://www.typescriptlang.org/ (accessed 29 October 2026).

14. PostgreSQL Global Development Group, PostgreSQL documentation; Redgate, Flyway documentation. [Online]. Available: https://www.postgresql.org/docs/ and https://documentation.red-gate.com/flyway (accessed 29 October 2026).

15. Leaflet, an open-source JavaScript library for interactive maps. [Online]. Available: https://leafletjs.com/ (accessed 29 October 2026).

16. Render and Neon hosting documentation. [Online]. Available: https://render.com/docs and https://neon.com/docs (accessed 29 October 2026).

Appendix

Appendix A: Project structure

Module	Contents
raktsetu-domain	Domain model: blood components, facilities, inventory, compatibility, interfaces, exceptions
raktsetu-engine	Allocation, expiry risk, data-freshness beliefs, demand forecasting, redistribution strategies
raktsetu-persistence	CSV repositories and the hash-chained audit file
raktsetu-sim	Digital-twin simulator and the forecasting experiment runner
raktsetu-cli	The 16-option console application
raktsetu-server	Spring Boot web server: REST API, security, database, alerts, facility directory
raktsetu-bench	JMH engine benchmarks and the HTTP load test
web	React and TypeScript front end, built into the server JAR
docs	UML diagrams, design notes, research method and results, benchmarks, security and privacy reviews
tools/directory	Scripts that build the national facility directory from NHP and OpenStreetMap data

Appendix B: Console application menu

Option	What it does
1	View one centre's stock, with units expiring within two days marked URGENT
2	View stock across the whole network, and groups running low
3	Add blood units
4	Raise a blood request (emergency or routine)
5	Check donor and recipient compatibility for a component
6	Run the expiry-risk check
7	Generate a redistribution plan
8	Compare the two strategies
9	Approve or reject proposed transfers
10	View the audit log and verify its hash chain
11	Run the 30-day simulation
12	View the wastage summary
13	Export reports as CSV

14	Data-freshness report
15	Set the date the app treats as today
16	Save and exit

Appendix C: Test results by module

Module	Tests	Result
raktsetu-domain	57	All pass
raktsetu-engine	44	All pass
raktsetu-persistence	17	All pass
raktsetu-sim	22	All pass
raktsetu-cli	7	All pass
raktsetu-server	80	All pass
raktsetu-bench	7	All pass
web (Vitest)	6	All pass
Total	240	Build success, 29 October 2026

Appendix D: Database migrations

Migration	Purpose
V1	Core schema: facilities, routes, units, usage, wastage, audit log
V2	Users and roles
V3	Workflows: requests, request units, transfers, notifications
V4	Real-site settings
V5	Alert outbox for SMS and WhatsApp
V6	Privacy controls
V7	Account requests (sign-up)
V8 to V12	National facility directory: tables, data load and reload

Appendix E: Technologies used

Tool	Used for
Java 21	The whole engine: object-oriented model, planning, simulator
Maven	Building the seven modules, running tests, packaging the release
Spring Boot and Spring Security	Web server, REST API, sign-in, roles and approval
PostgreSQL (Neon) and H2	Live database; built-in database for the console app and demo

Flyway	Versioned database changes
React and TypeScript (Vite)	The web front end
Leaflet and OpenStreetMap	Facilities map and the national directory
JUnit 5, Vitest, JaCoCo	Automated tests and the coverage gate
JMH	Engine benchmarks
Docker and Render	Packaging and hosting the live site
GitHub Actions	Builds and tests every change
PlantUML	UML diagrams

Appendix F: Source code and links

- Live application: https://raktsetu-ubvx.onrender.com
- Source code: https://github.com/pritikakhandelwal/raktsetu
- UML sources: docs/uml/*.puml in the repository
- Research method and results: docs/research/forecasting/ in the repository
- Windows release (console and web app): GitHub release v1.1.0